

Log-Prime Encoded Optical Logic and Multiplexing System

Using LCD Spectral Masking, Optical NOR Logic, Single-Detector Decoding, and Two-Dimensional Intensity/Voltage Readout

Document field	Value
Document type	Defensive technical disclosure
Inventor / author	Sebastian Shepherd
Publication purpose	Timestamped public disclosure for authorship attribution and prior-art style defensibility
Original disclosure date	30 July 2026
Recommended licence	CC BY 4.0 or CC BY-SA 4.0 for the paper; optional CERN-OHL-S or TAPR OHL for hardware files

Defensive publication statement: This document is intended to publicly disclose the described log-prime optical encoding, decoding, and demonstration architecture so that the author can credibly cite the work and so that the disclosed subject matter may be discoverable as public technical prior art. It is not a granted patent and does not itself provide exclusionary patent rights. Publishing may affect later patentability. Legal advice is recommended before publication if patent rights are desired.

Abstract

This disclosure describes a log-prime optical encoding and decoding method, together with an author-devised free-space experimental architecture for demonstrating the method. The disclosed system may be applied to wavelength-division multiplexed (WDM) optical systems, optical logic demonstrations, and related aggregate photodetector readout systems. For avoidance of doubt, this disclosure does not claim invention of WDM, amplitude-division multiplexing (ADM), diffraction gratings, LCD masking, photodetectors, lenses, or optical logic gates themselves. Those are established technical areas. The disclosed contribution is the use of a prime-derived logarithmic frequency/code rule, together with a single-detector decoding method, that allows combinations of log-prime-defined optical channel states to be represented and recovered from mean-like or weighted-mean detector readout states, optionally assisted by a separate amplitude or current measurement.

A preferred experimental embodiment independently developed by the author uses a broadband white LED, collimating optics, a diffraction grating, a convex focusing lens, an LCD spectral mask, free-space lens routing, an optical NOR stage, and a single photodetector receiver. In this embodiment, the horizontal spectral dimension of the LCD is used to select optical lanes whose intended frequency positions are generated from a log-prime relation rather than from arbitrary or equally spaced channel positions. Each intended channel frequency, or calibrated frequency-code value, is generated from a prime number P according to:

$$f(P) = A * \log(P) + B$$

where A and B are fixed positive constants in the chosen discrete code scale or calibrated frequency model, and P is a prime number. The resulting frequency values are converted into target wavelengths and then mapped, in the proposed grating, lens, and LCD geometry, to LCD pixel columns. The vertical or transmissive dimension of the LCD may optionally control amplitude.

In a constant-intensity mode, active lanes are decoded from an arithmetic mean of their calibrated voltage states. In a variable-intensity mode, the voltage reading is treated as an amplitude-weighted mean, and a simultaneous amplitude/current measurement supplies a second decoding axis.

The accompanying software proof of concept demonstrates generation of a calculated log-prime wavelength-to-pixel mapping for the proposed grating, lens, and LCD geometry, encoding of selected channels into LCD masks, simulated mapped-lane optical NOR behaviour, and recovery of the resulting active lane subset from a simulated mean-voltage-code measurement using a product-tree prime-factor decoder. In the recorded demonstration, a 12-lane calculated optical-core map was generated over a 495 to 740 nm cropped wavelength window and the decoded NOR vector matched the simulated mapped-lane NOR output.

Keywords: log-prime encoding; log-prime frequency mapping; wavelength-channel coding; single-detector decoding; optical state encoding; WDM implementation; prime-factor coding; optical NOR demonstration; LCD spectral masking; amplitude-weighted voltage mean; two-dimensional optical readout

1. Field of the Disclosure

The disclosure relates to encoding and decoding methods for optical communication, optical computing, and free-space optical demonstration systems. More particularly, it relates to optical systems in which intended wavelength or frequency-channel positions are generated according to a prime-derived logarithmic rule and decoded from aggregate photodetector measurements. The disclosed method may be implemented using existing wavelength-separation, optical logic, spatial-light-modulation, lens-projection, LCD-masking, and photodetector systems.

2. Prior-Art Positioning and Author Contribution

WDM, ADM, optical logic gates, diffraction-grating spectral separation, LCD or spatial-light-modulator masking, lens projection, photodiode detection, and analog current-to-voltage readout are established areas

of optical engineering. The author does not claim to have originated those underlying technologies or components.

The author contribution disclosed here includes both the log-prime optical frequency/code rule and the author-devised implementation architecture for demonstrating that rule. In the disclosed method, usable optical channel positions are not merely labelled after selection. Instead, intended channel frequencies or calibrated frequency-code values are generated from prime numbers according to a relation of the form:

$$f(P) = A * \log(P) + B$$

where P is a prime number and A and B are fixed positive constants in the chosen code or calibration scale. The resulting frequencies may then be converted into wavelengths and mapped onto LCD pixel columns using the proposed grating, lens, and LCD geometry. The prime number is therefore not merely an after-the-fact label for an existing wavelength lane. It is used to generate the intended frequency or code position of that lane.

The preferred embodiment developed by the author uses established optical components in a specific free-space arrangement: broadband illumination, diffraction-grating spectral separation, lens projection, LCD spectral masking, lens routing, optical NOR behaviour, and single-detector voltage/current readout. This implementation architecture is part of the disclosure. The disclosure does not claim ownership of basic WDM, ADM, diffraction gratings, lenses, LCDs, photodetectors, or optical logic generally, but does claim authorship of the disclosed log-prime frequency-generation method, decoding method, and system architecture.

3. Background and Motivation

Conventional wavelength-multiplexed optical systems often separate or process multiple optical lanes using filters, gratings, demultiplexers, coherent receivers, multiple detectors, or other lane-specific receiver hardware. For low-cost experimentation, teaching, telemetry, and proof-of-principle optical logic demonstrations, it is useful to encode multiple optical channel states so that a single detector or a small number of electrical measurements can recover lane information.

A raw photodetector naturally integrates incident optical power over the detector area and over its spectral response. This makes the detector good at producing aggregate measurements, but it also means that channel identity can be lost unless the optical lanes are encoded in a recoverable way. The disclosed log-prime method treats aggregation as a useful physical operation rather than a problem to be avoided.

4. Why Log-Prime Encoding Is Used

The disclosed method uses a log-prime frequency/code pattern rather than merely assigning prime labels to pre-existing wavelength lanes. Each intended optical frequency or calibrated frequency-code value is generated from a prime number P according to:

$$f(P) = A * \log(P) + B$$

where P is a prime number, and A and B are fixed positive constants in the chosen discrete code scale or calibrated frequency model.

The important point is not simply that sums of logarithms can be made unique. Unique additive sums can be achieved more directly using ordinary binary-weighted encoding, such as 1, 2, 4, 8, 16, and so on. The reason for using log-prime encoding here is that the voltage readout is treated as a mean-like code-state measurement rather than as a simple summed-voltage measurement.

For a set of active lanes, each generated from a distinct prime p_i , the mean log-prime code state is:

$$M = (\log(p_1) + \log(p_2) + \dots + \log(p_N)) / N$$

$$M = \log((p_1 * p_2 * \dots * p_N)^{(1/N)})$$

$$\exp(M) = (p_1 * p_2 * \dots * p_N)^{(1/N)}$$

Thus, the measured mean corresponds to the geometric mean of the active prime-defined lane identities.

Because the active lane product is formed from distinct prime factors, unique prime factorisation makes the mean log-prime value unique for each active subset under the distinct-prime constant-intensity model. If two active subsets with sizes m and n produced the same mean log-prime value, then their prime products would satisfy:

$$(\text{product_A})^n = (\text{product_B})^m$$

For square-free products of distinct primes, unique prime factorisation requires the same prime support and the same active count. Therefore, under the distinct-prime constant-intensity model, different active subsets do not produce the same mean log-prime code state, assuming ideal arithmetic and sufficient measurement precision.

This is the key distinction from ordinary binary weighting. Binary weighting is naturally suited to unique summed-current or summed-intensity codes. Log-prime encoding is useful here because it gives uniqueness in a mean-code domain, which is closer to how the voltage readout is interpreted in this disclosure.

The electrical readout should therefore be separated into two cases. In the constant-intensity case, where each active lane contributes approximately equally, the measured voltage-code state may be treated as the arithmetic mean of the active calibrated lane states:

$$V_{\text{measured}} = (V_1 + V_2 + \dots + V_N) / N$$

$$V_i = A * \log(p_i) + B$$

In the variable-intensity case, the voltage-code state becomes an amplitude-weighted mean:

$$V_{\text{measured}} = \text{sum}(V_i * A_i) / \text{sum}(A_i)$$

where A_i is the amplitude, intensity, or current contribution of lane i . In this variable-intensity case, voltage alone may not uniquely identify the active subset, because different amplitude combinations can shift the weighted mean. The separate amplitude or current measurement is therefore used as an additional constraint. Meanwhile, current or total amplitude remains additive:

$$I_{\text{total}} = \text{sum}(I_i)$$

This gives two complementary readout axes. The voltage channel provides the mean or weighted-mean log-prime code state, while the current or amplitude channel provides total optical intensity information.

Example: if the active lanes are generated from primes 2, 5, and 7, the pure log-prime mean is:

$$M = (\log(2) + \log(5) + \log(7)) / 3$$

$$M = \log((2 * 5 * 7)^{(1/3)})$$

$$M = \log(70^{(1/3)})$$

If M is the normalised log-prime mean after subtracting offset and dividing by gain, a decoder can test possible active counts N and reconstruct the corresponding candidate product:

$$\text{product_guess} = \exp(N * M)$$

For the correct active count $N = 3$, the recovered product is 70, which factorises uniquely as $70 = 2 * 5 * 7$. Therefore, the active lanes are identified as the lanes generated from primes 2, 5, and 7. Practical reliability depends on calibration, detector linearity, ADC resolution, voltage noise, intensity equality or amplitude compensation, wavelength-dependent optical response, and spacing between valid mean-code states.

5. Physics of Voltage, Current, and Detector Readout

A voltmeter measures electric potential difference, commonly interpreted as energy per unit charge. An ammeter measures current, or charge flow per unit time. In a photodetector receiver, incident light creates charge carriers, and the resulting electrical signal may be measured as direct photocurrent, voltage across a load, or voltage from a transimpedance amplifier.

For the purposes of the disclosed encoding framework, the voltage channel is treated as a calibrated code-state axis and the current or amplitude channel is treated as an intensity axis. A real implementation must calibrate source spectrum, LCD transmission, grating efficiency, lens loss, detector responsivity, dark current, ambient light, gain, offset, and noise.

Quantity	Meaning in this disclosure	Role
P_i	Optical power passing through lane i	Determines charge-carrier contribution
A_i or I_i	Amplitude/current contribution of lane i	Weighting term for variable-intensity decoding
V_i	Calibrated voltage-code state for the lane generated from prime p_i	Represents $A * \log(p_i) + B$ in the voltage-code domain
V_{measured}	Measured voltage or ADC value	Mean or weighted mean of active code states
A_{total} or I_{total}	Measured total amplitude/current	Second readout axis for ADM-like operation
$\text{sum}(V_i * A_i) / \text{sum}(A_i)$	Weighted mean model	Variable-intensity decoding relation

6. Two-Dimensional Encoding: Wavelength State and Amplitude State

The same demonstration bench can encode information in two dimensions. The horizontal spectral dimension selects wavelength lanes whose intended frequency positions are generated by the log-prime rule and then mapped onto LCD columns by the proposed grating/lens/LCD geometry. The vertical LCD dimension, transmissive area, duty cycle, or greyscale transparency may control how much optical power passes through each lane. This optional second dimension is an amplitude or intensity state layered on top of wavelength identity.

This disclosure does not claim to invent ADM itself. Rather, it discloses how an amplitude/intensity readout can be combined with log-prime wavelength encoding to produce a two-dimensional electrical decoding problem.

$$A_i = b_0 * 1 + b_1 * 2 + b_2 * 4 + b_3 * 8 + \dots$$

If amplitude states are binary-weighted or otherwise uniquely weighted, the current or total-amplitude channel can narrow the possible amplitude configuration. The voltage channel then identifies the weighted average of the active log-prime lane states.

7. Decoder Interpretation and Constraint Solving

The preceding sections define the two decoder regimes: a constant-intensity regime, where the voltage-code readout is treated as a mean log-prime state, and a variable-intensity regime, where the voltage-code readout is treated as an amplitude-weighted mean.

In the constant-intensity regime, the decoder first normalises the measured voltage by subtracting offset and dividing by gain:

$$M_{\text{log}} = (V_{\text{measured}} - B) / A$$

where M_{log} is the recovered mean log-prime code value. The decoder then tests possible active lane counts N and estimates the corresponding prime product:

`product_guess = exp(N * M_log)`

If the guessed product factorises into valid primes from the generated lane set, and the reconstructed voltage lies within the allowed tolerance, the decoder returns the corresponding active lanes.

In the variable-intensity regime, voltage alone may not uniquely identify the active subset because different amplitude combinations can shift the weighted mean. The decoder therefore uses the measured amplitude or current as an additional constraint:

`A_total = sum(A_i)`

Measured pair: (V_measured, A_total)

The measured pair is compared against a calibrated set of valid lane and amplitude combinations. The decoder selects the combination whose predicted voltage and amplitude most closely match the measured values within tolerance. This section does not introduce a new encoding method. It describes how the receiver interprets the voltage and amplitude measurements defined earlier and searches for the lane combination generated by the log-prime frequency/code rule.

8. Preferred Experimental System Overview

The preferred experimental arrangement independently devised for this disclosure uses the following optical and electrical chain: white LED, collimating lens, diffraction grating, convex focusing lens, LCD panel for input masking, lens routing, optical NOR gate, photodetector, voltmeter or ADC, and optionally an ammeter or current-sensing circuit.

White LED -> collimating lens -> diffraction grating -> convex lens -> LCD spectral mask -> lens routing -> optical NOR gate -> photodetector -> voltmeter/ADC and optional current readout

This arrangement is disclosed as the author's preferred experimental implementation architecture for the log-prime encoding and decoding method. The underlying components, including the LED, lenses, diffraction grating, LCD panel, photodetector, and electrical measurement devices, are not claimed as individually novel. The disclosed contribution includes their author-devised arrangement within this specific log-prime optical encoding system.

9. Log-Prime Wavelength Mapping Method

The log-prime wavelength mapping method begins with prime numbers, not with pre-existing arbitrary wavelength lanes. For each selected prime P, the software generates an intended frequency or calibrated frequency-code value using the log-prime relation:

`frequency_THz = A_thz * log2(P) + B_thz`

where A_thz and B_thz are constants for the selected optical window and calibration model, and P is a prime number.

The generated frequency is then converted into a target wavelength. That target wavelength is mapped through the proposed grating/lens/LCD geometry onto an LCD x-pixel position. The software then snaps the ideal floating-point pixel position to the nearest usable integer LCD column, checks for duplicate columns, verifies pixel spacing, and records snapping error.

This means the prime number is not merely an after-the-fact label for an existing wavelength lane. The prime number is used to generate the intended frequency or frequency-code position, and the proposed optical geometry is then used to calculate where that generated wavelength would fall on the LCD.

Mapped field	Purpose
channel	Logical identifier for the generated log-prime lane
prime	Prime used to generate the intended frequency/code position

log2_prime	Log-prime coordinate used for fitting
target_wavelength_nm	Ideal wavelength from the fitted log-prime model
integer_pixel	Calculated LCD x-column used in the proposed optical mapping
snap_error_px	Difference between ideal floating-pixel location and integer column
actual_pixel_wavelength_nm	Modelled wavelength corresponding to the selected LCD column
pixel_left/right_boundary_nm	Approximate modelled wavelength interval for that column

10. LCD Spectral Masking and Input Encoding

The LCD is positioned at or near the spectral image plane produced by the diffraction grating and focusing lens, so that horizontal LCD position corresponds to wavelength in the proposed optical geometry. Opening a calculated LCD column passes the lane generated from the corresponding prime-defined frequency/code value, while closing the column blocks it.

Partial opening, vertical aperture control, greyscale opacity, or time duty cycle may provide amplitude control. This allows the LCD to act both as a spectral lane selector and, optionally, as an amplitude-control element.

The accompanying LCD codec encodes selected generated channels into machine-readable patterns, text masks, PBM preview images, and C arrays. It also decodes saved patterns back into generated channel identifiers by checking which calculated LCD columns are open.

11. Optical NOR Gate

The disclosed demonstration includes an optical NOR stage after input encoding and routing. The functional condition is that a calculated output lane is optically high only when the corresponding input lanes are absent or below threshold.

In the software proof of concept, the simulated mapped-lane NOR rule is:

$$\text{OUT}[x] = \text{mapped}(x) \text{ AND NOT}(\text{INPUT1}[x] \text{ OR } \text{INPUT2}[x])$$

Here, mapped(x) means that x is one of the LCD columns calculated from the generated log-prime wavelength map. Unmapped LCD columns are not treated as optical channels. They are forced to zero in the simulated optical-core output.

This correction is important because a raw 128-column NOR over all LCD columns would falsely treat unused LCD columns as real wavelength lanes. In the disclosed model, only columns produced by the calculated log-prime wavelength-to-pixel mapping participate in the optical-core logic.

12. Detector Readout and Decoding Method

A photodetector converts the combined optical output into an electrical signal. Depending on the circuit, the measured quantity may be direct photocurrent, voltage across a resistor, voltage from a transimpedance amplifier, or a digitised ADC value. A slow proof of concept may use a voltmeter or logged ADC reading.

The voltage readout is interpreted as a calibrated code-state measurement. In the constant-intensity case, it may be treated as a mean log-prime voltage-code state. In the variable-intensity case, it is treated as an amplitude-weighted mean. A separate current or amplitude measurement may provide an additional constraint.

A practical decoder can follow this workflow:

1. Acquire detector voltage and, where used, current or amplitude measurement.
2. Subtract dark offset and background illumination estimate.
3. Apply gain, wavelength-response, LCD-transmission, and detector calibration.

4. Normalise the voltage reading into a mean log-prime code value, where applicable: $M_{\log} = (V_{\text{measured}} - B) / A$.
5. In the constant-intensity case, test possible active lane counts N and reconstruct candidate products: $\text{product_guess} = \exp(N * M_{\log})$.
6. Check whether the candidate product factorises into valid primes from the generated log-prime lane set.
7. In the variable-intensity case, additionally compare the measured amplitude or current against the expected amplitude state: $A_{\text{total}} = \sum(A_i)$.
8. Select the lane and amplitude combination whose predicted voltage and amplitude lie within the allowed error windows.
9. Return the active generated lanes and any associated amplitude state.

Accuracy is limited by ADC effective number of bits, detector linearity, dynamic range, shot noise, thermal noise, ambient light leakage, source stability, grating/lens alignment, LCD contrast ratio, calibration drift, intensity mismatch, and spacing between valid mean-code states.

13. Software Proof-of-Concept Implementation

The author developed a software pipeline that models and validates the log-prime encoding concept. The software records the mapping method, optical-geometry assumptions, logical conventions, decoder model, and reproducible example output for the disclosure.

The software proof of concept should be understood as a calculated and simulated validation of the disclosed method. It is not, by itself, an experimentally calibrated physical wavelength measurement. Physical calibration of a realised bench would still be required to confirm exact wavelength-to-pixel positions and measured detector responses.

File	Role in disclosure	Good insertion point
Wavelength_LCD_Encoder.py	Generates the calculated log-prime wavelength-to-pixel map and fitted optical equation.	Appendix B and Section 9
optical_lcd_codec.py	Encodes selected generated channels into LCD masks and decodes generated masks back to channel identifiers.	Appendix C and Section 10
random_input.py	Generates repeatable random mapped input patterns for Input 2.	Appendix D and Section 14
nor.py	Runs a raw 128-column NOR for debug only. It does not know which LCD columns correspond to generated log-prime lanes.	Appendix E note only
main.py	Runs the end-to-end simulated optical-core pipeline and overwrites raw NOR with the calculated mapped-lane NOR output.	Section 14 and Appendix E
nor_voltage_decoder.py	Decodes active generated channels from a simulated mean-voltage-code measurement using product reconstruction and a product tree.	Section 12, Section 14, Appendix F

14. Recorded Software Demonstration Output

The following recorded run demonstrates a 12-lane scalar-readout software model of the disclosed log-prime optical core. This is a software proof of concept and should be presented separately from physical bench measurements unless corresponding physical voltages and optical calibration values are later added.

Result field	Recorded value
Mode	cropped_window
Channels	12
Lens	100.0 mm
Wavelength window	495.000 to 740.000 nm

Mapping equation	$\text{frequency_THz} = A * \log_2(P) + B$
A	47.634780251 THz
B	357.490162992 THz
Integer pixel range	0 to 110
Minimum integer gap	2 px
Maximum snap error	0.3723 px
Prime product log10	12.870
Calculated mapped LCD pixels	0, 4, 6, 12, 17, 20, 28, 33, 49, 63, 87, 110

Input / output item	Recorded value
Input 1 active channels	1, 5, 12
Input 2 active channels	1, 2, 6, 8, 10, 11
Input 2 seed	1234
Simulated NOR active mapped lanes	4
Decoded active channels	3, 4, 7, 9
Decoded active pixels	63, 49, 20, 12
Recovered active count	4
Simulated measured voltage	1.23810139054 V
Expected decoded voltage	1.23810139054 V
Absolute decode error	0 V
Decoded matches simulated mapped-lane NOR	True
Decode time	0.414424 s
Total time	14.877495 s

The practical scalar-readout estimate for a 12-lane core over a 3.3 V ADC range is 4096 possible states, approximately 805.7 microvolts per state, a minimum half-step accuracy of approximately +/-402.8 microvolts, and a safer quarter-step target of approximately +/-201.4 microvolts.

The strict mathematical tolerance for exact prime-product reconstruction in the recorded decoded subset was approximately +/-9.13e-7 V. This is a diagnostic proof metric for the product-reconstruction decoder and is not the same as the practical scalar-core voltage target.

For publication, the mapping summary and decode summary may be included in the main body. Full terminal output, generated configuration files, and source code should be placed in appendices or an accompanying public repository so that the disclosure remains readable.

15. Disclosed Embodiments and Variations

- A broadband or multi-wavelength source is separated into wavelength lanes using a diffraction grating, prism, filters, fibre components, or integrated photonic structures, and the intended lane frequencies or frequency-code positions are generated from prime numbers using a log-prime relation.
- An LCD, spatial light modulator, digital micromirror device, mask, shutter array, or equivalent component selectively transmits, blocks, attenuates, or modulates lanes corresponding to the generated log-prime frequency pattern.
- Horizontal spectral position selects wavelength identity while vertical aperture, greyscale, exposure time, or duty cycle optionally controls amplitude.
- A single detector or small detector set receives an aggregate optical output. Voltage, current, normalised ratios, or time-sampled values are used to infer generated lane state and, where applicable, amplitude state.
- Encoded optical signals may pass through an optical NOR stage before detection.

- The decoder may use lookup tables, nearest-code-point matching, active-count search, mean-log-prime product reconstruction, simultaneous-equation search, product-tree GCD extraction, or calibrated machine decoding.
- Alternative embodiments may use narrowband LEDs, lasers, prisms, filters, fibre components, spatial light modulators, digital micromirror devices, integrated photonic elements, or other wavelength-selective arrangements to implement the same log-prime frequency/code rule.

16. Proof-of-Concept Status and Required Experimental Record

The author reports a crude proof of concept and has provided a software validation run. The software validation demonstrates the calculated log-prime mapping, simulated mapped-lane NOR behaviour, and decoder recovery. For final public upload, physical measurement details should be added if available. If exact values are not available, this section should remain an honest experimental-status statement rather than an overclaim.

Record item	Purpose	Status
Number of wavelength lanes tested	Shows scale of demonstrated system	Software calculated map: 12 lanes; physical bench: to be filled by author
Photodetector model or type	Defines bandwidth and responsivity context	To be filled by author
Voltage/current measurement device	Defines readout resolution and repeatability	To be filled by author
Measured physical code-point values	Shows separation between lane combinations	To be filled by author if available
Software decode output	Shows reproducible mathematical proof of concept	Provided in Section 14
Photos or schematic	Supports provenance and reproducibility	Recommended

17. Limitations

- This disclosure is not a claim to a high-speed commercial WDM receiver as written. It is a log-prime encoding framework, decoding method, and low-cost proof-of-principle implementation architecture.
- WDM, ADM, LCD masking, photodetection, diffraction gratings, lenses, and optical NOR gates are not claimed as individually novel.
- The software mapping is a calculated wavelength-to-pixel mapping for a proposed optical geometry. Physical calibration would be required to confirm exact wavelength positions, detector response, optical losses, and code-point separations in a realised bench setup.
- The raw detector is primarily an integrating device. Mean and two-dimensional interpretations require calibration, normalisation, known amplitude states, equal-intensity assumptions, amplitude compensation, or separate total-current measurement.
- In the variable-intensity case, voltage alone may not uniquely identify the active subset because weighted means can collide. The current or amplitude channel is therefore used as an additional constraint.
- White LED spectrum, LCD transmission, grating efficiency, lens aberration, alignment, and detector responsivity vary with wavelength and require calibration.
- The exact number of reliably distinguishable states depends on noise, ADC resolution, detector linearity, dynamic range, temperature drift, source stability, and valid-code spacing.
- Publishing this disclosure may affect later patentability in jurisdictions where prior public disclosure is novelty-destroying.

18. Potential Applications

- Low-cost educational demonstrations of WDM concepts, optical logic, wavelength coding, optical masking, log-prime frequency mapping, and analog decoding.
- Optical sensor multiplexing where multiple generated frequency/code states are read with minimal detector hardware.
- Bench-top experiments in optical computing, spatial light modulation, aggregate detector coding, and detector-integrated optical readout.
- Telemetry or state-signalling systems where lane identity can be encoded into mean-like voltage/code states and optionally combined with a current or amplitude readout.
- Software/hardware co-design demonstrations for optical cores using calculated log-prime wavelength maps, programmable LCD masking, optical logic behaviour, and single-detector decoding.

19. Reciprocal Research Intent and Licensing

This disclosure is published to establish a timestamped public authorship record for the log-prime optical frequency/code generation and single-detector decoding system developed by Sebastian Shepherd. The author's intent is to make the work available for study, replication, validation, improvement, and further public research.

This publication is not a granted patent and does not itself provide exclusionary patent rights. Rather, it is intended to serve as a public technical disclosure and prior-art style record. Defensive publication can help make disclosed subject matter publicly discoverable as prior art, but it does not by itself create patent-style exclusive rights.

To encourage public contribution, the author intends the associated materials to be shared under reciprocal open licences:

- Paper, diagrams, explanatory text, and documentation: Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0).

- Software source code: GNU Affero General Public License v3.0 or later (AGPL-3.0-or-later).

- Hardware design files, schematics, CAD files, optical bench diagrams, and related design documentation: CERN Open Hardware Licence Strongly Reciprocal v2.0 (CERN-OHL-S-2.0).

These licences are intended to preserve attribution to the author and to encourage adaptations, improvements, software modifications, hardware variants, physical replications, calibration data, and derived research outputs to remain publicly available under reciprocal terms where the licences apply.

Anyone who builds on this work is requested to cite the original disclosure and, where possible, publish improvements, experimental results, calibration data, modified code, or derived hardware documentation in a public repository or archival release.

For citation and CV use, the public repository should include a CITATION.cff file and a DOI-bearing archival release, so that the paper, codebase, output files, and disclosure record can be cited consistently.

20. Code Theory Test output

```
py -3.13 main.py --force-map --input1-channels 1,5,12 --input2-active-count 6 --seed 1234 --skip-external-nor
```

```
=====
=====
```

1. Generate wavelength/log-prime LCD map

```
=====
=====
```

```
$ C:\Users\sebsh\AppData\Local\Programs\Python\Python313\python.exe Wavelength_LCD_Encoder.py
```

```
=====
=
```

Mode: cropped_window

Channels: 12

Lens: 100.0 mm

Wavelength window: 495.000 to 740.000 nm

Equation: $\text{frequency_THz} = A * \log_2(P) + B$

A: 47.634780251 THz

B: 357.490162992 THz

Integer pixel range: 0 to 110

Integer pixel span: 110 px

Minimum integer gap: 2 px

Max snap error: 0.3723 px

Duplicate pixels: False

Fits LCD: True

Passes integer gaps: True

Passes snap error: True

VALID: True

=====

First 10 mapped integer columns:

px= 0 ch=12 P=37 targetlambda=495.000nm actuallambda=495.000nm snap=+0.000px gap-boundary=493.781-496.218nm

px= 4 ch=11 P=31 targetlambda=505.141nm actuallambda=504.721nm snap=-0.173px gap-boundary=503.509-505.933nm

px= 6 ch=10 P=29 targetlambda=509.073nm actuallambda=509.561nm snap=+0.202px gap-boundary=508.353-510.769nm

px= 12 ch= 9 P=23 targetlambda=523.226nm actuallambda=523.995nm snap=+0.321px gap-boundary=522.797-525.192nm

px= 17 ch= 8 P=19 targetlambda=535.497nm actuallambda=535.924nm snap=+0.179px gap-boundary=534.735-537.112nm

px= 20 ch= 7 P=17 targetlambda=542.910nm actuallambda=543.037nm snap=+0.054px gap-boundary=541.854-544.220nm

px= 28 ch= 6 P=13 targetlambda=561.662nm actuallambda=561.845nm snap=+0.078px gap-boundary=560.676-563.013nm

px= 33 ch= 5 P=11 targetlambda=574.008nm actuallambda=573.479nm snap=-0.228px gap-boundary=572.320-574.637nm

px= 49 ch= 4 P=7 targetlambda=610.304nm actuallambda=610.068nm snap=-0.105px gap-boundary=608.940-611.196nm

px= 63 ch= 3 P=5 targetlambda=640.453nm actuallambda=641.272nm snap=+0.372px gap-boundary=640.171-642.373nm

Last 10 mapped integer columns:

px= 6 ch=10 P=29 targetlambda=509.073nm actuallambda=509.561nm snap=+0.202px gap-boundary=508.353-510.769nm

px= 12 ch= 9 P=23 targetlambda=523.226nm actuallambda=523.995nm snap=+0.321px gap-boundary=522.797-525.192nm

px= 17 ch= 8 P=19 targetlambda=535.497nm actuallambda=535.924nm snap=+0.179px gap-boundary=534.735-537.112nm

px= 20 ch= 7 P=17 targetlambda=542.910nm actuallambda=543.037nm snap=+0.054px gap-boundary=541.854-544.220nm

px= 28 ch= 6 P=13 targetlambda=561.662nm actuallambda=561.845nm snap=+0.078px gap-boundary=560.676-563.013nm

px= 33 ch= 5 P=11 targetlambda=574.008nm actuallambda=573.479nm snap=-0.228px gap-boundary=572.320-574.637nm

px= 49 ch= 4 P=7 targetlambda=610.304nm actuallambda=610.068nm snap=-0.105px gap-boundary=608.940-611.196nm

px= 63 ch= 3 P=5 targetlambda=640.453nm actuallambda=641.272nm snap=+0.372px gap-boundary=640.171-642.373nm

px= 87 ch= 2 P=3 targetlambda=692.378nm actuallambda=692.964nm snap=+0.278px gap-boundary=691.910-694.016nm

px=110 ch= 1 P=2 targetlambda=740.000nm actuallambda=740.335nm snap=+0.166px gap-boundary=739.328-741.341nm

Prime product log10: 12.870

Saved CSV: integer_log_prime_wavelength_map.csv

Saved CFG: integer_log_prime_wavelength_map.cfg

Top candidates:

1.valid=True mode=cropped_window channels=12 lens=100 mm range=495.0-740.0nm px=0-110 gap=2 snap=0.372px log10P=12.87

2.valid=True mode=cropped_window channels=12 lens=100 mm range=420.0-713.5nm px=0-127 gap=2 snap=0.400px log10P=12.87

3.valid=True mode=cropped_window channels=12 lens=100 mm range=445.0-733.8nm px=0-127 gap=2 snap=0.407px log10P=12.87

4.valid=True mode=cropped_window channels=12 lens=80 mm range=500.0-740.0nm px=0-86 gap=2 snap=0.409px log10P=12.87

5.valid=True mode=cropped_window channels=12 lens=100 mm range=460.0-740.0nm px=0-124 gap=2
snap=0.415px log10P=12.87

6.valid=True mode=cropped_window channels=12 lens=100 mm range=510.0-740.0nm px=0-104 gap=2
snap=0.433px log10P=12.87

7.valid=True mode=cropped_window channels=12 lens=80 mm range=480.0-740.0nm px=0-93 gap=2
snap=0.435px log10P=12.87

8.valid=True mode=cropped_window channels=12 lens=100 mm range=415.0-709.5nm px=0-127 gap=2
snap=0.438px log10P=12.87

9.valid=True mode=cropped_window channels=12 lens=80 mm range=505.0-740.0nm px=0-85 gap=2
snap=0.439px log10P=12.87

10.valid=True mode=cropped_window channels=12 lens=100 mm range=450.0-737.9nm px=0-127 gap=2
snap=0.440px log10P=12.87

Mapped optical wavelength lanes: 12

Mapped LCD pixels: [0, 4, 6, 12, 17, 20, 28, 33, 49, 63, 87, 110]

=====

2. Generate Input 1 vector/pattern

=====

```
$ C:\Users\sebsb\AppData\Local\Programs\Python\Python313\python.exe optical_lcd_codec.py encode --  
map integer_log_prime_wavelength_map.cfg --channels 1,5,12 --out input1
```

Saved: input1.txt

Saved: input1.json

Saved: input1.pbm

Saved: input1.c

Encoded channels decoded back from generated pattern:

channel 1: amplitude 1, x=110

channel 5: amplitude 1, x=33

channel 12: amplitude 1, x=0

```
=====
=====
```

3. Generate random Input 2 vector/pattern

```
=====
=====
```

```
$ C:\Users\sebsb\AppData\Local\Programs\Python\Python313\python.exe random_input.py --map
integer_log_prime_wavelength_map.cfg --amp-min 1 --amp-max 1 --seed 1234 --out input2 --active-count 6
```

Saved input2.txt

Saved input2.pbm

Saved input2.json

Saved input2.c

Random Input 2 Summary

Mapped channels available: 12

Active channels selected: 6

Seed: 1234

Amplitude range: 1..1

Vertical mode: bottom

channel= 1 amp= 1 x=110 prime=2 wavelength_nm=740.3349297816112

channel= 2 amp= 1 x= 87 prime=3 wavelength_nm=692.9638059705146

channel= 6 amp= 1 x= 28 prime=13 wavelength_nm=561.8450145421746

channel= 8 amp= 1 x= 17 prime=19 wavelength_nm=535.923612897034

channel= 10 amp= 1 x= 6 prime=29 wavelength_nm=509.56110752575466

channel= 11 amp= 1 x= 4 prime=31 wavelength_nm=504.7214751622113

Skipping external NOR script; main.py will compute mapped NOR output.

=====

5. Decode unknown mean voltage back into active mapped channels

=====

=====

```
$ C:\Users\sebsbh\AppData\Local\Programs\Python\Python313\python.exe nor_voltage_decoder.py --map
integer_log_prime_wavelength_map.cfg --actual-nor nor_result.json --precision 5000 --voltage-tolerance
0.000001 --out unknown_decode_report.json --vector-out decoded_unknown_vector.txt --simulate-from-
actual
```

Product-Tree Unknown Mean-Voltage Decode

Mapped channels: 12

Recovered active count: 4

Measured voltage: 1.23810139054e0 V

Expected decoded voltage: 1.23810139054e0 V

Absolute error: 0 V

Candidate counts after prune: 11

Decimal exp attempts: 1

Factor attempts: 1

Product-tree extractions: 1

Decode time: 0.431163 s

Total time: 14.958729 s

Minimum Voltage Accuracy Required for Exact Prime Reconstruction

Diagnostic only

Prime product digits: 5

log10(product): 4.136245

Minimum voltage accuracy: +/-9.13392155890e-7 V

Positive-side safe error: +9.13392155890e-7 V

Negative-side safe error: -9.13425528518e-7 V

Meaning:

This is the strict mathematical tolerance needed for the decoder to reconstruct the exact prime product using $\text{round}(\exp(kL))$.

This is useful as a diagnostic/proof metric, but it is NOT the practical voltage target for reconstructing the 12-lane core output.

Decoded active channels: [3, 4, 7, 9]

Decoded active pixels: [63, 49, 20, 12]

Matches actual NOR vector: True

Saved decoded vector: decoded_unknown_vector.txt

Saved report: unknown_decode_report.json

Minimum Voltage Accuracy Required for Lane Reconstruction

Practical scenario

Optical core size: 12 NOR lanes

Possible lane output combinations: 4096

ADC / voltmeter range assumed: 3.3 V

Voltage spacing per lane state: 8.05664e-4 V

Minimum voltage accuracy: +/-4.02832e-4 V

Recommended safer target: +/-2.01416e-4 V

Meaning:

This is the practical hardware requirement for reconstructing the

lane-state output of the scalar optical core.

For a 12-lane core over 3.3 V:

$2^{12} = 4096$ possible lane output combinations

$3.3 \text{ V} / 4096 = \text{about } 805.7 \text{ uV per state}$

minimum half-step accuracy = about $\pm 402.8 \text{ uV}$

safer quarter-step target = about $\pm 201.4 \text{ uV}$

```
=====
=====
```

PIPELINE COMPLETE

```
=====
=====
```

Mapped optical lanes: 12

Input 1 active mapped lanes: 3

Input 2 active mapped lanes: 6

NOR active mapped lanes: 4

Decoded active mapped lanes: 4

Decoded matches mapped NOR: True

Wavelength map: integer_log_prime_wavelength_map.cfg

Input 1 JSON: input1.json

Input 2 JSON: input2.json

Mapped NOR JSON: nor_result.json

Decode report: unknown_decode_report.json

Decoded vector: decoded_unknown_vector.txt

PS D:\Desktop\Optical GPU driver>

21. Codebase

Note on Source-Code Wording

The source code below is included as a historical software proof-of-concept record. Some comments, variable names, command-line arguments, or printed output may use prototype terms such as “actual,” “physical,”

“mapped optical,” or “measured” for internal modelling purposes. Unless accompanied by separately recorded physical bench measurements, those terms should be read as referring to the calculated or simulated model described in the main disclosure.

The codebase is included to document the author’s implementation logic, mapping assumptions, decoder model, and reproducible theory-test output. It is not presented as a fully calibrated commercial optical receiver or as a substitute for physical wavelength, voltage, current, or detector-response calibration.

Main.py

```
#!/usr/bin/env python3
"""
main.py

End-to-end runner for the Optical GPU Driver proof-of-concept.

Important model fix:
- The optical system only has lanes at the mapped LCD wavelength columns.
- Unmapped LCD columns are not optical channels and must not participate in NOR.
- Therefore this runner computes a MAPPED NOR result:
    OUT[x] = 1 only if x is a mapped wavelength pixel AND input1[x] == 0 AND
input2[x] == 0
    OUT[x] = 0 for every unmapped pixel

Runs in order:
    1. Wavelength_LCD_Encoder.py      -> integer_log_prime_wavelength_map.cfg/.csv
    2. optical_lcd_codec.py encode     -> input1.json
    3. random_input.py                 -> input2.json
    4. nor.py                           -> optional external NOR run, mostly for debug
output
    5. main.py                         -> overwrites nor_result.* with mapped-channel NOR
    6. nor_voltage_decoder.py           -> decodes unknown mean voltage and compares to
mapped NOR

Run:
    py -3.13 main.py
"""

from __future__ import annotations

import argparse
import configparser
import csv
import json
import re
```

```

import subprocess
import sys
from pathlib import Path
from typing import Any, List, Set
from decimal import Decimal, getcontext, ROUND_CEILING

LANE_COUNT = 128

DEFAULT_MAP_SCRIPT = "Wavelength_LCD_Encoder.py"
DEFAULT_CODEC_SCRIPT = "optical_lcd_codec.py"
DEFAULT_RANDOM_SCRIPT = "random_input.py"
DEFAULT_NOR_SCRIPT = "nor.py"
DEFAULT_DECODER_SCRIPT = "nor_voltage_decoder.py"

DEFAULT_MAP_CFG = "integer_log_prime_wavelength_map.cfg"
DEFAULT_INPUT1_PREFIX = "input1"
DEFAULT_INPUT2_PREFIX = "input2"
DEFAULT_NOR_PREFIX = "nor_result"
DEFAULT_DECODE_REPORT = "unknown_decode_report.json"
DEFAULT_DECODED_VECTOR = "decoded_unknown_vector.txt"

def run_step(label: str, command: List[str], allow_fail: bool = False) ->
subprocess.CompletedProcess:
    print()
    print("=" * 100)
    print(label)
    print("=" * 100)
    print("$ " + " ".join(command))

    result = subprocess.run(
        command,
        text=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        encoding="utf-8",
        errors="replace",
    )

    if result.stdout:
        print(result.stdout, end="" if result.stdout.endswith("\n") else "\n")

    if result.stderr:
        print(result.stderr, end="" if result.stderr.endswith("\n") else "\n")

    if result.returncode != 0 and not allow_fail:
        raise RuntimeError(f"Step failed: {label} | exit={result.returncode}")

```

```

    return result

def require_file(path: str | Path, label: str) -> Path:
    file_path = Path(path)
    if not file_path.exists():
        raise FileNotFoundError(f"Missing {label}: {file_path}")
    return file_path

def clean_header(header: str) -> str:
    if header is None:
        return ""
    text = str(header).strip()
    text = text.replace('<strong data-lexical-text="true">', '')
    text = text.replace('</strong>', '')
    text = text.replace(',', '')
    return text.strip()

def load_mapped_pixels(map_path: str | Path) -> Set[int]:
    """
    Return the set of actual wavelength-channel LCD columns from the mapper output.
    These are the only columns that are real optical lanes.
    """
    path = Path(map_path)
    if not path.exists():
        raise FileNotFoundError(f"Map file not found: {path}")

    pixels: Set[int] = set()

    if path.suffix.lower() == ".cfg":
        cfg = configparser.ConfigParser()
        cfg.read(path, encoding="utf-8")

        for section in cfg.sections():
            if not section.lower().startswith("channel_"):
                continue
            pixels.add(int(cfg[section]["integer_pixel"]))

    elif path.suffix.lower() == ".csv":
        with path.open("r", newline="", encoding="utf-8-sig") as file:
            reader = csv.DictReader(file)
            for row in reader:
                clean = {clean_header(key): value for key, value in row.items()}
                pixels.add(int(clean["integer_pixel"]))
    else:
        raise ValueError("Map must be .cfg or .csv")

```

```

for pixel in pixels:
    if not 0 <= pixel < LANE_COUNT:
        raise ValueError(f"Mapped pixel {pixel} is outside 0..{LANE_COUNT - 1}")

if not pixels:
    raise ValueError("No mapped pixels found in map file.")

return pixels

def load_json(path: str | Path) -> Any:
    with Path(path).open("r", encoding="utf-8") as file:
        return json.load(file)

def save_json(path: str | Path, data: Any) -> None:
    with Path(path).open("w", encoding="utf-8") as file:
        json.dump(data, file, indent=2)

def compress_pattern_to_128(pattern: List[List[Any]]) -> List[int]:
    if not pattern:
        raise ValueError("Pattern is empty.")

    width = len(pattern[0])
    if width != LANE_COUNT:
        raise ValueError(f"Pattern width is {width}; expected {LANE_COUNT}.")

    for row_index, row in enumerate(pattern):
        if len(row) != LANE_COUNT:
            raise ValueError(f"Pattern row {row_index} has width {len(row)}; expected {LANE_COUNT}.")

    return [1 if any(float(pattern[y][x]) > 0 for y in range(len(pattern))) else 0 for
x in range(LANE_COUNT)]

def normalise_128(value: Any) -> List[int]:
    if not isinstance(value, list):
        raise ValueError("Expected list or 2D pattern.")

    if len(value) == LANE_COUNT and all(not isinstance(item, list) for item in value):
        return [1 if float(item) > 0 else 0 for item in value]

    if value and all(isinstance(row, list) for row in value):
        return compress_pattern_to_128(value)

```

```

    raise ValueError("Expected 128-number list or 2D pattern.")

def load_128(path: str | Path) -> List[int]:
    file_path = Path(path)
    if not file_path.exists():
        raise FileNotFoundError(f"Could not load 128-vector: {file_path}")

    if file_path.suffix.lower() == ".json":
        data = load_json(file_path)

        if isinstance(data, list):
            return normalise_128(data)

        if isinstance(data, dict):
            for key in ["output", "values", "lanes", "input", "input1", "input2"]:
                if key in data:
                    return normalise_128(data[key])

            if "pattern" in data:
                return compress_pattern_to_128(data["pattern"])

            raise ValueError(f"Unsupported JSON vector structure: {file_path}")

    text = file_path.read_text(encoding="utf-8").strip()

    if text.startswith("P1"):
        return load_pbm_128(text)

    tokens = [token for token in re.split(r"[\s,;]+", text) if token]
    values = [1 if float(token) > 0 else 0 for token in tokens]

    if len(values) != LANE_COUNT:
        raise ValueError(f"{file_path} produced {len(values)} values; expected {LANE_COUNT}.")

    return values

def load_pbm_128(text: str) -> List[int]:
    lines: List[str] = []
    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        lines.append(line)

    if not lines or lines[0] != "P1":

```



```

        raise ValueError("Invalid PBM data. Expected P1 header.")

width, height = [int(part) for part in lines[1].split()[:2]]
if width != LANE_COUNT:
    raise ValueError(f"PBM width is {width}; expected {LANE_COUNT}.")

tokens: List[str] = []
for line in lines[2:]:
    tokens.extend(line.split())

values = [1 if float(token) > 0 else 0 for token in tokens]
if len(values) != width * height:
    raise ValueError(f"PBM has {len(values)} pixels; expected {width * height}.")

if height == 1:
    return values

pattern: List[List[int]] = []
for y in range(height):
    start = y * width
    pattern.append(values[start:start + width])

return compress_pattern_to_128(pattern)

def mask_to_mapped_lanes(values: List[int], mapped_pixels: Set[int]) -> List[int]:
    """
    Force all unmapped columns to zero.
    """
    if len(values) != LANE_COUNT:
        raise ValueError("Vector must have 128 lanes.")
    return [1 if index in mapped_pixels and values[index] else 0 for index in
range(LANE_COUNT)]

def mapped_nor_128(input1: List[int], input2: List[int], mapped_pixels: Set[int]) ->
List[int]:
    """
    NOR only over real optical wavelength lanes.
    Unmapped columns are not physical lanes, so they always output zero.
    """
    if len(input1) != LANE_COUNT or len(input2) != LANE_COUNT:
        raise ValueError("Both inputs must have exactly 128 lanes.")

    output = [0 for _ in range(LANE_COUNT)]

    for pixel in mapped_pixels:
        output[pixel] = 1 if input1[pixel] == 0 and input2[pixel] == 0 else 0

```

```

    return output

def save_128_txt(path: str | Path, values: List[int]) -> None:
    Path(path).write_text(" ".join(str(value) for value in values) + "\n",
encoding="utf-8")

def save_nor_result(prefix: str, input1: List[int], input2: List[int], output:
List[int], mapped_pixels: Set[int]) -> None:
    active_indices = [index for index, value in enumerate(output) if value == 1]

    save_128_txt(f"{prefix}.txt", output)
    Path(f"{prefix}.csv").write_text(",".join(str(value) for value in output) + "\n",
encoding="utf-8")

    save_json(
        f"{prefix}.json",
        {
            "lane_count": LANE_COUNT,
            "logic": "MAPPED_OUT[x] = mapped(x) AND NOT(INPUT1[x] OR INPUT2[x])",
            "convention": "0=inactive/no-light/false, 1=active/light-present/true",
            "mapped_pixels": sorted(mapped_pixels),
            "input1": input1,
            "input2": input2,
            "output": output,
            "active_output_indices": active_indices,
        },
    )

    with Path(f"{prefix}.c").open("w", encoding="utf-8") as file:
        file.write("#include <stdint.h>\n\n")
        file.write(f"const uint8_t nor_output[{LANE_COUNT}] = {{\n")
        for index in range(0, LANE_COUNT, 16):
            chunk = output[index:index + 16]
            file.write("    " + ", ".join(str(value) for value in chunk))
            if index + 16 < LANE_COUNT:
                file.write(",")
            file.write("\n")
        file.write("};\n")

def decimal_scientific(value: Decimal, significant_digits: int = 6) -> str:
    """Format a Decimal in compact scientific notation."""
    if value == 0:
        return "0"

```

```

sign = "-" if value < 0 else ""
value = abs(value)
exponent = value.adjusted()
mantissa = value.scaleb(-exponent)

# Keep display short but stable.
shown = f"{mantissa:.{significant_digits - 1}f}"
return f"{sign}{shown}e{exponent}"

def compute_readout_accuracy_for_effective_bits(
    effective_bits: int,
    adc_range_volts: str = "3.3",
) -> dict:
    """
    Compute the actual scalar-readout voltage accuracy required for a core
    with a fixed number of effective output bits.

    This is the physically relevant calculation for the practical core model.

    For a core with b independent binary NOR lanes:

        possible states = 2 ** b

    If those states are spread across an ADC/voltmeter range V_range:

        voltage_step = V_range / (2 ** b)

    To decode safely by nearest voltage state:

        minimum required accuracy = voltage_step / 2

    For engineering margin, a conservative target is:

        recommended accuracy = voltage_step / 4

    Example for b=12 and V_range=3.3 V:

        states = 4096
        step = 805.664  $\mu$ V
        required =  $\pm$ 402.832  $\mu$ V
        recommended =  $\pm$ 201.416  $\mu$ V
    """

    if effective_bits <= 0:
        raise ValueError("effective_bits must be greater than zero.")

```

```

adc_range = Decimal(str(adc_range_volts))
state_count = Decimal(2) ** Decimal(effective_bits)

voltage_step = adc_range / state_count
required_abs_voltage = voltage_step / Decimal(2)
recommended_abs_voltage = voltage_step / Decimal(4)

return {
    "effective_bits": effective_bits,
    "state_count": int(state_count),
    "adc_range_volts": adc_range,
    "voltage_step": voltage_step,
    "required_abs_voltage": required_abs_voltage,
    "recommended_abs_voltage": recommended_abs_voltage,
}
def print_core_readout_accuracy(accuracy: dict) -> None:
    print()
    print("Minimum Voltage Accuracy Required for Lane Reconstruction")
    print("Practical scenario")
    print("-----")
    print(f"Optical core size: {accuracy['effective_bits']} NOR
lanes")
    print(f"Possible lane output combinations: {accuracy['state_count']}")
    print(f"ADC / voltmeter range assumed: {accuracy['adc_range_volts']} V")
    print()
    print(f"Voltage spacing per lane
state: {decimal_scientific(accuracy['voltage_step'], 6)} V")
    print(f"Minimum voltage accuracy: +/-
{decimal_scientific(accuracy['required_abs_voltage'], 6)} V")
    print(f"Recommended safer target: +/-
{decimal_scientific(accuracy['recommended_abs_voltage'], 6)} V")
    print()
    print("Meaning:")
    print(" This is the practical hardware requirement for reconstructing the")
    print(" lane-state output of the scalar optical core.")
    print(" For a 12-lane core over 3.3 V:")
    print(" 2^12 = 4096 possible lane output combinations")
    print(" 3.3 V / 4096 = about 805.7 uV per state")
    print(" minimum half-step accuracy = about +/-402.8 uV")
    print(" safer quarter-step target = about +/-201.4 uV")
def build_random_command(args: argparse.Namespace) -> List[str]:
    command = [
        sys.executable,
        args.random_script,
        "--map",
        args.map_cfg,
        "--amp-min",
        "1",

```

```

        "--amp-max",
        "1",
        "--seed",
        str(args.seed),
        "--out",
        args.input2_out,
    ]

    if args.input2_active_probability is not None:
        command += ["--active-probability", str(args.input2_active_probability)]
    else:
        command += ["--active-count", str(args.input2_active_count)]

    return command

def build_decoder_command(args: argparse.Namespace) -> List[str]:
    command = [
        sys.executable,
        args.decoder_script,
        "--map",
        args.map_cfg,
        "--actual-nor",
        f"{args.nor_out}.json",
        "--precision",
        str(args.precision),
        "--voltage-tolerance",
        str(args.voltage_tolerance),
        "--out",
        args.decode_out,
        "--vector-out",
        args.decoded_vector_out,
    ]

    if args.measured_voltage is None:
        command += ["--simulate-from-actual"]
    else:
        command += ["--measured-voltage", str(args.measured_voltage)]

    return command

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="Run the full Optical GPU Driver proof pipeline.")

    parser.add_argument("--map-script", default="Wavelength_LCD_Encoder.py")
    parser.add_argument("--codec-script", default="optical_lcd_codec.py")

```

```

parser.add_argument("--random-script", default="random_input.py")
parser.add_argument("--nor-script", default="nor.py")
parser.add_argument("--decoder-script", default="nor_voltage_decoder.py")

parser.add_argument("--map-cfg", default="integer_log_prime_wavelength_map.cfg")
parser.add_argument("--force-map", action="store_true", help="Regenerate
wavelength map even if cfg already exists.")

parser.add_argument(
    "--input1-channels",
    default="1,5,12",
    help="Input 1 channels, e.g. 1,5,12. Must exist in the generated map."
)

parser.add_argument("--input1-out", default="input1")

parser.add_argument("--input2-out", default="input2")
parser.add_argument(
    "--input2-active-count",
    type=int,
    default=6,
    help="Number of random active Input 2 channels. For a 12-lane core, keep this <=
12."
)

parser.add_argument("--input2-active-probability", type=float, default=None)
parser.add_argument("--seed", type=int, default=1234)

parser.add_argument("--nor-out", default="nor_result")
parser.add_argument("--skip-external-nor", action="store_true", help="Do not call
nor.py; main.py computes mapped NOR itself.")

parser.add_argument("--measured-voltage", default=None, help="Actual measured
voltage. If absent, decoder simulates from actual mapped NOR.")
parser.add_argument("--precision", type=int, default=5000)
parser.add_argument("--voltage-tolerance", default="0.000001")
parser.add_argument("--decode-out", default="unknown_decode_report.json")
parser.add_argument("--decoded-vector-out", default="decoded_unknown_vector.txt")
parser.add_argument("--adc-range-volts", default="3.3", help="ADC/voltmeter full-
scale range used for ideal bits estimate.")
parser.add_argument(
    "--effective-bits",
    type=int,
    default=12,
    help="Effective output bits per scalar-readout core. Default: 12."
)

return parser

```

```

def main() -> None:
    args = build_parser().parse_args()

    require_file(args.map_script, "wavelength mapper script")
    require_file(args.codec_script, "LCD codec script")
    require_file(args.random_script, "random input script")
    require_file(args.decoder_script, "mean-voltage decoder script")

    if args.force_map or not Path(args.map_cfg).exists():
        run_step("1. Generate wavelength/log-prime LCD map", [sys.executable,
args.map_script])
    else:
        print(f"Using existing wavelength map: {args.map_cfg}")

    require_file(args.map_cfg, "wavelength map cfg")
    mapped_pixels = load_mapped_pixels(args.map_cfg)

    print()
    print(f"Mapped optical wavelength lanes: {len(mapped_pixels)}")
    print(f"Mapped LCD pixels: {sorted(mapped_pixels)}")

    run_step(
        "2. Generate Input 1 vector/pattern",
        [
            sys.executable,
            args.codec_script,
            "encode",
            "--map",
            args.map_cfg,
            "--channels",
            args.input1_channels,
            "--out",
            args.input1_out,
        ],
    )

    run_step("3. Generate random Input 2 vector/pattern", build_random_command(args))

    input1_json = f"{args.input1_out}.json"
    input2_json = f"{args.input2_out}.json"
    require_file(input1_json, "Input 1 json")
    require_file(input2_json, "Input 2 json")

    if not args.skip_external_nor:
        require_file(args.nor_script, "NOR script")
        run_step(
            "4. Run external NOR script for debug",
            [

```

```

        sys.executable,
        args.nor_script,
        "--input1",
        input1_json,
        "--input2",
        input2_json,
        "--out",
        args.nor_out,
    ],
)
else:
    print("Skipping external NOR script; main.py will compute mapped NOR output.")

raw_input1 = load_128(input1_json)
raw_input2 = load_128(input2_json)

input1 = mask_to_mapped_lanes(raw_input1, mapped_pixels)
input2 = mask_to_mapped_lanes(raw_input2, mapped_pixels)
mapped_nor = mapped_nor_128(input1, input2, mapped_pixels)

# This intentionally overwrites nor_result.* so the decoder sees the physical
optical NOR result,
# not the logical complement of all 128 LCD columns.
save_nor_result(args.nor_out, input1, input2, mapped_nor, mapped_pixels)

run_step("5. Decode unknown mean voltage back into active mapped channels",
build_decoder_command(args))

decoded_vector = load_128(args.decoded_vector_out)
decoded_vector = mask_to_mapped_lanes(decoded_vector, mapped_pixels)
matches = decoded_vector == mapped_nor

accuracy = compute_readout_accuracy_for_effective_bits(
    effective_bits=args.effective_bits,
    adc_range_volts=args.adc_range_volts,
)
print_core_readout_accuracy(accuracy)

print()
print("=" * 100)
print("PIPELINE COMPLETE")
print("=" * 100)
print(f"Mapped optical lanes:      {len(mapped_pixels)}")
print(f"Input 1 active mapped lanes: {sum(input1)}")
print(f"Input 2 active mapped lanes: {sum(input2)}")
print(f"NOR active mapped lanes:     {sum(mapped_nor)}")
print(f"Decoded active mapped lanes: {sum(decoded_vector)}")
print(f"Decoded matches mapped NOR:  {matches}")

```



```

print(f"Wavelength map:           {args.map_cfg}")
print(f"Input 1 JSON:             {input1_json}")
print(f"Input 2 JSON:             {input2_json}")
print(f"Mapped NOR JSON:         {args.nor_out}.json")
print(f"Decode report:            {args.decode_out}")
print(f"Decoded vector:           {args.decoded_vector_out}")

if not matches:
    raise RuntimeError("Decoded vector did not match mapped optical NOR output.")

if __name__ == "__main__":
    main()

```

Wavelength_LCD_Encoder.py

```

from __future__ import annotations
import math
import csv
from dataclasses import dataclass
import configparser

C = 299_792_458.0

@dataclass
class OpticalConfig:
    led_min_nm: float = 380.0
    led_max_nm: float = 740.0

    lcd_pixels_x: int = 128
    lcd_width_mm: float = 21.5

    grating_lines_per_mm: float = 600.0
    diffraction_order: int = 1

    lens_options_mm: tuple = (80.0, 100.0)

    min_channels: int = 32
    max_channels: int = 64

    # Integer gap between actual selected LCD columns.
    # 1 means adjacent columns allowed.
    # 2 means at least one blank/guard column between channels.
    min_integer_pixel_gap: int = 2

```

```

# Maximum allowed snapping error from ideal log-prime wavelength position
# to chosen integer LCD pixel.
max_snap_error_px: float = 0.5

allow_cropped_windows: bool = True
crop_step_nm: float = 1.0

def is_prime_64bit(n: int) -> bool:
    if n < 2:
        return False

    small_primes = [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
        37, 41, 43, 47
    ]

    for p in small_primes:
        if n == p:
            return True
        if n % p == 0:
            return False

    d = n - 1
    s = 0

    while d % 2 == 0:
        s += 1
        d //= 2

    for a in [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]:
        if a >= n:
            continue

        x = pow(a, d, n)

        if x == 1 or x == n - 1:
            continue

        passed = False

        for _ in range(s - 1):
            x = pow(x, 2, n)

            if x == n - 1:
                passed = True
                break

```

```

        if not passed:
            return False

    return True

def next_prime(n: int) -> int:
    if n <= 2:
        return 2

    if n % 2 == 0:
        n += 1

    while not is_prime_64bit(n):
        n += 2

    return n

def prime_ladder(channel_count: int) -> list:
    """
    Return the first `channel_count` raw prime numbers.

    This minimises the prime product size compared with the old power-of-two ladder.

    Old:
        p_i ~= next_prime(2 ** i)

    New:
        p_i = ith prime

    This reduces the required voltage precision for the mean-voltage decoder,
    but it also makes wavelength positions bunch together more, so fewer channels
    may fit without duplicate LCD pixels.
    """

    primes: list[int] = []
    candidate = 2

    while len(primes) < channel_count:
        if is_prime_64bit(candidate):
            primes.append(candidate)

        candidate += 1

    return primes

```

```

def frequency_thz_from_wavelength_nm(wavelength_nm: float) -> float:
    return (C / (wavelength_nm * 1e-9)) / 1e12

def wavelength_nm_from_frequency_thz(frequency_thz: float) -> float:
    return (C / (frequency_thz * 1e12)) * 1e9

def grating_x_position_mm(
    wavelength_nm: float,
    focal_length_mm: float,
    grating_lines_per_mm: float,
    diffraction_order: int
) -> float:
    d_mm = 1.0 / grating_lines_per_mm
    wavelength_mm = wavelength_nm * 1e-6

    argument = diffraction_order * wavelength_mm / d_mm

    if argument < -1.0 or argument > 1.0:
        raise ValueError(
            f"Wavelength {wavelength_nm} nm is invalid for order "
            f"{diffraction_order} with {grating_lines_per_mm} lines/mm."
        )

    theta_rad = math.asin(argument)
    return focal_length_mm * math.tan(theta_rad)

def wavelength_to_pixel_float(
    wavelength_nm: float,
    anchor_wavelength_nm: float,
    anchor_pixel: float,
    focal_length_mm: float,
    config: OpticalConfig
) -> float:
    pixel_pitch_mm = config.lcd_width_mm / config.lcd_pixels_x

    x = grating_x_position_mm(
        wavelength_nm=wavelength_nm,
        focal_length_mm=focal_length_mm,
        grating_lines_per_mm=config.grating_lines_per_mm,
        diffraction_order=config.diffraction_order
    )

    x_anchor = grating_x_position_mm(
        wavelength_nm=anchor_wavelength_nm,
        focal_length_mm=focal_length_mm,

```

```

        grating_lines_per_mm=config.grating_lines_per_mm,
        diffraction_order=config.diffraction_order
    )

    return anchor_pixel + (x - x_anchor) / pixel_pitch_mm

def pixel_to_wavelength_nm(
    pixel: float,
    anchor_wavelength_nm: float,
    anchor_pixel: float,
    focal_length_mm: float,
    config: OpticalConfig
) -> float:
    pixel_pitch_mm = config.lcd_width_mm / config.lcd_pixels_x

    x_anchor = grating_x_position_mm(
        wavelength_nm=anchor_wavelength_nm,
        focal_length_mm=focal_length_mm,
        grating_lines_per_mm=config.grating_lines_per_mm,
        diffraction_order=config.diffraction_order
    )

    x = x_anchor + (pixel - anchor_pixel) * pixel_pitch_mm
    theta_rad = math.atan(x / focal_length_mm)

    d_mm = 1.0 / config.grating_lines_per_mm
    wavelength_mm = (d_mm / config.diffraction_order) * math.sin(theta_rad)

    return wavelength_mm * 1e6

def fit_log_prime_equation(
    primes: list[int],
    wavelength_min_nm: float,
    wavelength_max_nm: float
) -> tuple[float, float]:
    """
    Fit exact endpoint equation:

        frequency_THz = A * log2(P) + B

    First prime maps to wavelength_max_nm.
    Last prime maps to wavelength_min_nm.
    """
    f_low_thz = frequency_thz_from_wavelength_nm(wavelength_max_nm)
    f_high_thz = frequency_thz_from_wavelength_nm(wavelength_min_nm)

```

```

log_first = math.log2(primes[0])
log_last = math.log2(primes[-1])

A_thz = (f_high_thz - f_low_thz) / (log_last - log_first)
B_thz = f_low_thz - A_thz * log_first

return A_thz, B_thz

def prime_product_log10(rows: list[dict]) -> float:
    """
    Estimate log10(product of all selected channel primes).

    Smaller is better because smaller prime products reduce the voltage accuracy
    required by the mean-voltage product reconstruction decoder.
    """

    total = 0.0

    for row in rows:
        total += math.log10(row["prime"])

    return total

def first_n_primes(count: int) -> list[int]:
    """
    Return the first `count` raw prime numbers.
    """
    primes: list[int] = []
    candidate = 2

    while len(primes) < count:
        if is_prime_64bit(candidate):
            primes.append(candidate)
            candidate += 1

    return primes

def prime_product_log10_from_primes(primes: list[int]) -> float:
    return sum(math.log10(prime) for prime in primes)

def make_rows_for_primes(
    primes: list[int],
    wavelength_min_nm: float,
    wavelength_max_nm: float,
    focal_length_mm: float,
    config: OpticalConfig

```

```

) -> tuple[list[dict], float, float]:
    """
    Map each prime to an LCD pixel using the fitted log-prime wavelength model.
    """
    A_thz, B_thz = fit_log_prime_equation(
        primes=primes,
        wavelength_min_nm=wavelength_min_nm,
        wavelength_max_nm=wavelength_max_nm
    )

    rows = []

    for channel, prime in enumerate(primes, start=1):
        log2_prime = math.log2(prime)

        target_frequency_thz = A_thz * log2_prime + B_thz
        target_wavelength_nm = wavelength_nm_from_frequency_thz(target_frequency_thz)

        float_pixel = wavelength_to_pixel_float(
            wavelength_nm=target_wavelength_nm,
            anchor_wavelength_nm=wavelength_min_nm,
            anchor_pixel=0.0,
            focal_length_mm=focal_length_mm,
            config=config
        )

        integer_pixel = int(round(float_pixel))

        actual_wavelength_nm = pixel_to_wavelength_nm(
            pixel=integer_pixel,
            anchor_wavelength_nm=wavelength_min_nm,
            anchor_pixel=0.0,
            focal_length_mm=focal_length_mm,
            config=config
        )

        actual_frequency_thz = frequency_thz_from_wavelength_nm(actual_wavelength_nm)

        rows.append({
            "channel": channel,
            "prime": prime,
            "log2_prime": log2_prime,
            "target_frequency_thz": target_frequency_thz,
            "target_wavelength_nm": target_wavelength_nm,
            "float_pixel": float_pixel,
            "integer_pixel": integer_pixel,
            "snap_error_px": integer_pixel - float_pixel,
            "actual_pixel_wavelength_nm": actual_wavelength_nm,

```

```

        "actual_pixel_frequency_thz": actual_frequency_thz,
        "wavelength_error_nm": actual_wavelength_nm - target_wavelength_nm,
        "frequency_error_thz": actual_frequency_thz - target_frequency_thz,
        "pixel_left_boundary_nm": pixel_to_wavelength_nm(
            pixel=integer_pixel - 0.5,
            anchor_wavelength_nm=wavelength_min_nm,
            anchor_pixel=0.0,
            focal_length_mm=focal_length_mm,
            config=config
        ),
        "pixel_right_boundary_nm": pixel_to_wavelength_nm(
            pixel=integer_pixel + 0.5,
            anchor_wavelength_nm=wavelength_min_nm,
            anchor_pixel=0.0,
            focal_length_mm=focal_length_mm,
            config=config
        )
    })

    return rows, A_thz, B_thz

def select_small_prime_subset(
    channel_count: int,
    wavelength_min_nm: float,
    wavelength_max_nm: float,
    focal_length_mm: float,
    config: OpticalConfig,
    pool_size: int | None = None
) -> list[int]:
    """
    Select a small-prime subset that maps to unique LCD pixels.
    """
    if pool_size is None:
        pool_size = max(256, channel_count * 16)

    prime_pool = first_n_primes(pool_size)

    best_primes: list[int] | None = None
    best_score: tuple | None = None

    first_prime = prime_pool[0]

    for endpoint_index in range(channel_count - 1, len(prime_pool)):
        endpoint_prime = prime_pool[endpoint_index]

        endpoint_primes = [first_prime, endpoint_prime]

```



```

A_thz, B_thz = fit_log_prime_equation(
    primes=endpoint_primes,
    wavelength_min_nm=wavelength_min_nm,
    wavelength_max_nm=wavelength_max_nm
)

pixel_to_prime: dict[int, tuple[int, float]] = {}

for prime in prime_pool[:endpoint_index + 1]:
    log2_prime = math.log2(prime)
    target_frequency_thz = A_thz * log2_prime + B_thz
    target_wavelength_nm =
wavelength_nm_from_frequency_thz(target_frequency_thz)

    try:
        float_pixel = wavelength_to_pixel_float(
            wavelength_nm=target_wavelength_nm,
            anchor_wavelength_nm=wavelength_min_nm,
            anchor_pixel=0.0,
            focal_length_mm=focal_length_mm,
            config=config
        )
    except ValueError:
        continue

    integer_pixel = int(round(float_pixel))
    snap_error = abs(integer_pixel - float_pixel)

    if integer_pixel < 0 or integer_pixel >= config.lcd_pixels_x:
        continue

    if snap_error > config.max_snap_error_px:
        continue

    existing = pixel_to_prime.get(integer_pixel)
    if existing is None or prime < existing[0]:
        pixel_to_prime[integer_pixel] = (prime, snap_error)

candidate_items = [
    {"pixel": pixel, "prime": prime, "snap_error": snap_error}
    for pixel, (prime, snap_error) in pixel_to_prime.items()
]

if len(candidate_items) < channel_count:
    continue

candidate_items.sort(key=lambda item: item["prime"])

```

```

selected: list[dict] = []

def pixel_ok(pixel: int) -> bool:
    return all(abs(pixel - item["pixel"]) >= config.min_integer_pixel_gap for
item in selected)

forced_primes = {first_prime, endpoint_prime}

for prime in forced_primes:
    forced_matches = [item for item in candidate_items if item["prime"] ==
prime]
    if not forced_matches:
        break
    item = forced_matches[0]
    if pixel_ok(item["pixel"]):
        selected.append(item)

if len(selected) != len(forced_primes):
    continue

for item in candidate_items:
    if item["prime"] in forced_primes:
        continue
    if pixel_ok(item["pixel"]):
        selected.append(item)
    if len(selected) >= channel_count:
        break

if len(selected) < channel_count:
    continue

selected = selected[:channel_count]
selected_primes = sorted(item["prime"] for item in selected)

if first_prime not in selected_primes:
    continue
if endpoint_prime not in selected_primes:
    continue

product_log10 = prime_product_log10_from_primes(selected_primes)
max_prime = max(selected_primes)
pixel_span = max(item["pixel"] for item in selected) - min(item["pixel"] for
item in selected)

score = (product_log10, max_prime, -pixel_span)

if best_score is None or score < best_score:
    best_score = score

```

```

        best_primes = selected_primes

    if best_primes is None:
        raise ValueError("Could not find a valid small-prime subset for this optical
window.")

    return best_primes

def generate_integer_channel_map(
    channel_count: int,
    wavelength_min_nm: float,
    wavelength_max_nm: float,
    focal_length_mm: float,
    config: OpticalConfig
) -> dict:
    if channel_count > config.lcd_pixels_x:
        raise ValueError("Channel count cannot exceed horizontal LCD pixel count.")

    primes = select_small_prime_subset(
        channel_count=channel_count,
        wavelength_min_nm=wavelength_min_nm,
        wavelength_max_nm=wavelength_max_nm,
        focal_length_mm=focal_length_mm,
        config=config
    )

    rows, A_thz, B_thz = make_rows_for_primes(
        primes=primes,
        wavelength_min_nm=wavelength_min_nm,
        wavelength_max_nm=wavelength_max_nm,
        focal_length_mm=focal_length_mm,
        config=config
    )

    rows_by_pixel = sorted(rows, key=lambda r: r["integer_pixel"])

    integer_pixels = [row["integer_pixel"] for row in rows_by_pixel]
    unique_pixels = sorted(set(integer_pixels))

    duplicate_pixels = len(unique_pixels) != len(integer_pixels)

    integer_gaps = [
        unique_pixels[i + 1] - unique_pixels[i]
        for i in range(len(unique_pixels) - 1)
    ]

    min_integer_gap = min(integer_gaps) if integer_gaps else 999999
    max_snap_error = max(abs(row["snap_error_px"]) for row in rows)

```

```

fits_lcd = (
    min(integer_pixels) >= 0
    and max(integer_pixels) <= config.lcd_pixels_x - 1
)

passes_integer_gap = min_integer_gap >= config.min_integer_pixel_gap
passes_snap_error = max_snap_error <= config.max_snap_error_px

valid = (
    fits_lcd
    and not duplicate_pixels
    and passes_integer_gap
    and passes_snap_error
)

return {
    "channel_count": channel_count,
    "wavelength_min_nm": wavelength_min_nm,
    "wavelength_max_nm": wavelength_max_nm,
    "focal_length_mm": focal_length_mm,

    "A_thz": A_thz,
    "B_thz": B_thz,

    "min_integer_pixel": min(integer_pixels),
    "max_integer_pixel": max(integer_pixels),
    "integer_pixel_span": max(integer_pixels) - min(integer_pixels),

    "min_integer_gap": min_integer_gap,
    "max_snap_error_px": max_snap_error,

    "duplicate_pixels": duplicate_pixels,
    "fits_lcd": fits_lcd,
    "passes_integer_gap": passes_integer_gap,
    "passes_snap_error": passes_snap_error,
    "valid": valid,

    "prime_product_log10_all_channels": prime_product_log10_from_primes(primes),
    "max_prime": max(primes),
    "min_prime": min(primes),

    "rows": rows_by_pixel
}

```

```

def max_wavelength_that_fits_from_min(
    wavelength_min_nm: float,
    focal_length_mm: float,

```

```

    config: OpticalConfig
) -> float:
    return pixel_to_wavelength_nm(
        pixel=config.lcd_pixels_x - 1,
        anchor_wavelength_nm=wavelength_min_nm,
        anchor_pixel=0.0,
        focal_length_mm=focal_length_mm,
        config=config
    )

def find_candidates(config: OpticalConfig) -> list:
    candidates = []

    for focal_length_mm in config.lens_options_mm:
        for channel_count in range(config.min_channels, config.max_channels + 1):
            try:
                full = generate_integer_channel_map(
                    channel_count=channel_count,
                    wavelength_min_nm=config.led_min_nm,
                    wavelength_max_nm=config.led_max_nm,
                    focal_length_mm=focal_length_mm,
                    config=config
                )
                full["mode"] = "full_led_range"
                candidates.append(full)
            except ValueError:
                pass

    if config.allow_cropped_windows:
        start_nm = config.led_min_nm

        while start_nm < config.led_max_nm:
            end_nm = max_wavelength_that_fits_from_min(
                wavelength_min_nm=start_nm,
                focal_length_mm=focal_length_mm,
                config=config
            )

            end_nm = min(end_nm, config.led_max_nm)

            if end_nm <= start_nm + 10:
                start_nm += config.crop_step_nm
                continue

            for channel_count in range(config.min_channels, config.max_channels +
1):
                try:

```

```

        cropped = generate_integer_channel_map(
            channel_count=channel_count,
            wavelength_min_nm=start_nm,
            wavelength_max_nm=end_nm,
            focal_length_mm=focal_length_mm,
            config=config
        )
        cropped["mode"] = "cropped_window"
        candidates.append(cropped)
    except ValueError:
        pass

    start_nm += config.crop_step_nm

def score_valid(result: dict) -> tuple:
    return (
        result["channel_count"],
        result["min_integer_gap"],
        -result["max_snap_error_px"],
        -result.get("prime_product_log10_all_channels", 0.0),
        result["integer_pixel_span"]
    )

def score_invalid(result: dict) -> tuple:
    return (
        result["channel_count"],
        -int(result["duplicate_pixels"]),
        int(result["fits_lcd"]),
        int(result["passes_snap_error"]),
        result["integer_pixel_span"]
    )

valid_candidates = [candidate for candidate in candidates if candidate["valid"]]
invalid_candidates = [candidate for candidate in candidates if not
candidate["valid"]]

valid_candidates.sort(key=score_valid, reverse=True)
invalid_candidates.sort(key=score_invalid, reverse=True)

return valid_candidates + invalid_candidates

def print_result(result: dict) -> None:
    print()
    print("=" * 90)
    print(f"Mode:                {result['mode']}")
    print(f"Channels:              {result['channel_count']}")
    print(f"Lens:                  {result['focal_length_mm']} mm")

```

```

    print(f"Wavelength window:      {result['wavelength_min_nm']:.3f} to
{result['wavelength_max_nm']:.3f} nm")
    print(f"Equation:                frequency_THz = A * log2(P) + B")
    print(f"A:                      {result['A_thz']:.9f} THz")
    print(f"B:                      {result['B_thz']:.9f} THz")
    print(f"Integer pixel range:      {result['min_integer_pixel']] to
{result['max_integer_pixel']}")
    print(f"Integer pixel span:       {result['integer_pixel_span']] px")
    print(f"Minimum integer gap:      {result['min_integer_gap']] px")
    print(f"Max snap error:           {result['max_snap_error_px']:.4f} px")
    print(f"Duplicate pixels:          {result['duplicate_pixels']}")
    print(f"Fits LCD:                 {result['fits_lcd']}")
    print(f"Passes integer gaps:       {result['passes_integer_gap']}")
    print(f"Passes snap error:          {result['passes_snap_error']}")
    print(f"VALID:                     {result['valid']}")
    print("=" * 90)

    print()
    print("First 10 mapped integer columns:")
    for row in result["rows"][:10]:
        print(
            f"px={row['integer_pixel']:>3}  "
            f"ch={row['channel']:>2}  "
            f"P={row['prime']}  "
            f"targetlambda={row['target_wavelength_nm']:.3f}nm  "
            f"actuellambda={row['actual_pixel_wavelength_nm']:.3f}nm  "
            f"snap={row['snap_error_px']:+.3f}px  "
            f"gap-boundary={row['pixel_left_boundary_nm']:.3f}-
{row['pixel_right_boundary_nm']:.3f}nm"
        )

    print()
    print("Last 10 mapped integer columns:")
    for row in result["rows"][-10:]:
        print(
            f"px={row['integer_pixel']:>3}  "
            f"ch={row['channel']:>2}  "
            f"P={row['prime']}  "
            f"targetlambda={row['target_wavelength_nm']:.3f}nm  "
            f"actuellambda={row['actual_pixel_wavelength_nm']:.3f}nm  "
            f"snap={row['snap_error_px']:+.3f}px  "
            f"gap-boundary={row['pixel_left_boundary_nm']:.3f}-
{row['pixel_right_boundary_nm']:.3f}nm"
        )

def save_csv(result: dict, filename: str) -> None:
    fieldnames = [

```

```

        "channel",
        "prime",
        "log2_prime",
        "target_frequency_thz",
        "target_wavelength_nm",
        "float_pixel",
        "integer_pixel",
        "snap_error_px",
        "actual_pixel_wavelength_nm",
        "actual_pixel_frequency_thz",
        "wavelength_error_nm",
        "frequency_error_thz",
        "pixel_left_boundary_nm",
        "pixel_right_boundary_nm"
    ]

    with open(filename, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()

        for row in result["rows"]:
            writer.writerow(row)

    print(f"\nSaved CSV: {filename}")

def save_cfg(result: dict, filename: str) -> None:
    """
    Save the generated integer log-prime wavelength map as a .cfg file.

    The file stores:
    - global optical setup
    - fitted log-prime equation constants
    - integer LCD pixel mapping
    - per-channel prime/wavelength/frequency data

    Output equation:
        frequency_THz = A_thz * log2(P) + B_thz
    """

    cfg = configparser.ConfigParser()

    cfg["setup"] = {
        "mode": str(result.get("mode", "unknown")),
        "channel_count": str(result["channel_count"]),
        "focal_length_mm": str(result["focal_length_mm"]),
        "wavelength_min_nm": str(result["wavelength_min_nm"]),
        "wavelength_max_nm": str(result["wavelength_max_nm"]),
        "min_integer_pixel": str(result["min_integer_pixel"]),
    }

```



```

    "max_integer_pixel": str(result["max_integer_pixel"]),
    "integer_pixel_span": str(result["integer_pixel_span"]),
    "min_integer_gap": str(result["min_integer_gap"]),
    "max_snap_error_px": str(result["max_snap_error_px"]),
    "duplicate_pixels": str(result["duplicate_pixels"]),
    "fits_lcd": str(result["fits_lcd"]),
    "passes_integer_gap": str(result["passes_integer_gap"]),
    "passes_snap_error": str(result["passes_snap_error"]),
    "valid": str(result["valid"])
}

cfg["log_prime_equation"] = {
    "equation": "frequency_THz = A_thz * log2(P) + B_thz",
    "A_thz": str(result["A_thz"]),
    "B_thz": str(result["B_thz"])
}

for row in result["rows"]:
    section_name = f"channel_{row['channel']:03d}"

    cfg[section_name] = {
        "channel": str(row["channel"]),
        "prime": str(row["prime"]),
        "log2_prime": str(row["log2_prime"]),

        "target_frequency_thz": str(row["target_frequency_thz"]),
        "target_wavelength_nm": str(row["target_wavelength_nm"]),

        "float_pixel": str(row["float_pixel"]),
        "integer_pixel": str(row["integer_pixel"]),
        "snap_error_px": str(row["snap_error_px"]),

        "actual_pixel_wavelength_nm": str(row["actual_pixel_wavelength_nm"]),
        "actual_pixel_frequency_thz": str(row["actual_pixel_frequency_thz"]),

        "wavelength_error_nm": str(row["wavelength_error_nm"]),
        "frequency_error_thz": str(row["frequency_error_thz"]),

        "pixel_left_boundary_nm": str(row["pixel_left_boundary_nm"]),
        "pixel_right_boundary_nm": str(row["pixel_right_boundary_nm"])
    }

with open(filename, "w", encoding="utf-8") as file:
    cfg.write(file)

print(f"Saved CFG: {filename}")

```

```

def main():
    config = OpticalConfig(
        led_min_nm=380.0,
        led_max_nm=740.0,

        lcd_pixels_x=128,
        lcd_width_mm=21.5,

        grating_lines_per_mm=600.0,
        diffraction_order=1,

        lens_options_mm=(80.0, 100.0),

        # 12-lane scalar-readout core.
        # This keeps one optical core around 12 effective bits / 4096 states.
        min_channels=12,
        max_channels=12,

        # Use 1 for aggressive proof-of-concept density.
        # Use 2 if you want guard pixels for a more forgiving physical setup.
        min_integer_pixel_gap=1,
        max_snap_error_px=0.5,

        allow_cropped_windows=True,

        # Faster search. Use 1.0 only for final map polishing.
        crop_step_nm=5.0
    )

    candidates = find_candidates(config)

    if not candidates:
        raise RuntimeError("No candidates were generated at all.")

    best = candidates[0]

    if not best["valid"]:
        print()
        print("=" * 90)
        print("NO VALID 12-LANE MAP FOUND")
        print("=" * 90)
        print("The best candidate was still invalid, so the mapper will not save it.")
        print()
        print_result(best)

        print()
        print("Try:")
        print("  1. min_integer_pixel_gap=1")

```

```

    print(" 2. crop_step_nm=1.0 for a finer search")
    print(" 3. a different lens option")
    print(" 4. a smaller wavelength window")
    print("=" * 90)

    raise RuntimeError("No valid 12-lane wavelength map found.")

print_result(best)

print()
print(f"Prime product log10: {best.get('prime_product_log10_all_channels',
0.0):.3f}")

save_csv(best, "integer_log_prime_wavelength_map.csv")
save_cfg(best, "integer_log_prime_wavelength_map.cfg")

print()
print("Top candidates:")
for i, candidate in enumerate(candidates[:10], start=1):
    print(
        f"{i:>2}. "
        f"valid={candidate['valid']} "
        f"mode={candidate['mode']:<16} "
        f"channels={candidate['channel_count']:<2} "
        f"lens={candidate['focal_length_mm']:<5.0f}mm "
        f"range={candidate['wavelength_min_nm']:.1f}-
{candidate['wavelength_max_nm']:.1f}nm "
        f"px={candidate['min_integer_pixel']}-{candidate['max_integer_pixel']} "
        f"gap={candidate['min_integer_gap']} "
        f"snap={candidate['max_snap_error_px']:.3f}px "
        f"log10P={candidate.get('prime_product_log10_all_channels', 0.0):.2f}"
    )
if __name__ == "__main__":
    main()

```

Random_input.py

```

#!/usr/bin/env python3
"""
random_input2_generator.py

Standalone random Input 2 pattern generator for the optical LCD core.

Purpose:
- Load your log-prime wavelength map from .cfg or .csv.
- Randomly choose channels for Input 2.
- Generate a 128x64 LCD pattern.
- Save the pattern as:
    input2_random.json machine-readable

```

```
input2_random.txt    human-readable
input2_random.pbm    monochrome PBM preview
input2_random.c      SSD1306-style byte array
```

Conventions:

- $x = 0..127$ = wavelength column axis.
- $y = 0..63$ = amplitude axis.
- 1 = open / pass light.
- 0 = closed / block light.

Examples:

```
python random_input2_generator.py --map integer_log_prime_wavelength_map.cfg --
active-count 8 --out input2_random
python random_input2_generator.py --map integer_log_prime_wavelength_map.csv --
active-probability 0.25 --seed 1234 --out input2_seeded
python random_input2_generator.py --map map.cfg --active-count 12 --amp-min 8 --
amp-max 64 --vertical-mode bottom
"""
```

```
import argparse
import configparser
import csv
import json
import random
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional

LCD_WIDTH = 128
LCD_HEIGHT = 1

@dataclass(frozen=True)
class ChannelInfo:
    channel: int
    prime: int
    log2_prime: float
    integer_pixel: int
    actual_pixel_wavelength_nm: Optional[float] = None
    actual_pixel_frequency_thz: Optional[float] = None

def clamp_int(value: int, low: int, high: int) -> int:
    return max(low, min(high, value))

def clean_header(header: str) -> str:
    """
```

```

Makes CSV loading tolerant of pasted rich-text headers.
"""
if header is None:
    return ""

text = str(header).strip()
text = text.replace('<strong data-lexical-text="true">', '')
text = text.replace('</strong>', '')
text = text.replace(',', ' ')
return text.strip()

def optional_float(mapping, key: str) -> Optional[float]:
    if key not in mapping:
        return None

    value = mapping[key]

    if value is None:
        return None

    value = str(value).strip()

    if value == "":
        return None

    return float(value)

def load_channel_map(path: str | Path) -> Dict[int, ChannelInfo]:
    path = Path(path)

    if not path.exists():
        raise FileNotFoundError(f"Map file not found: {path}")

    if path.suffix.lower() == ".cfg":
        channels = load_channel_map_cfg(path)
    elif path.suffix.lower() == ".csv":
        channels = load_channel_map_csv(path)
    else:
        raise ValueError("Map file must be .cfg or .csv")

    validate_channel_map(channels)
    return channels

def load_channel_map_cfg(path: Path) -> Dict[int, ChannelInfo]:
    cfg = configparser.ConfigParser()

```

```

cfg.read(path, encoding="utf-8")

channels: Dict[int, ChannelInfo] = {}

for section in cfg.sections():
    if not section.lower().startswith("channel_"):
        continue

    item = cfg[section]

    channel = int(item["channel"])
    prime = int(item["prime"])
    log2_prime = float(item["log2_prime"])
    integer_pixel = int(item["integer_pixel"])

    channels[channel] = ChannelInfo(
        channel=channel,
        prime=prime,
        log2_prime=log2_prime,
        integer_pixel=integer_pixel,
        actual_pixel_wavelength_nm=optional_float(item,
"actual_pixel_wavelength_nm"),
        actual_pixel_frequency_thz=optional_float(item,
"actual_pixel_frequency_thz"),
    )

return channels

def load_channel_map_csv(path: Path) -> Dict[int, ChannelInfo]:
    channels: Dict[int, ChannelInfo] = {}

    with path.open("r", newline="", encoding="utf-8-sig") as file:
        reader = csv.DictReader(file)

        for row in reader:
            clean = {clean_header(key): value for key, value in row.items()}

            channel = int(clean["channel"])
            prime = int(clean["prime"])
            log2_prime = float(clean["log2_prime"])
            integer_pixel = int(clean["integer_pixel"])

            channels[channel] = ChannelInfo(
                channel=channel,
                prime=prime,
                log2_prime=log2_prime,
                integer_pixel=integer_pixel,

```

```

        actual_pixel_wavelength_nm=optional_float(clean,
"actual_pixel_wavelength_nm"),
        actual_pixel_frequency_thz=optional_float(clean,
"actual_pixel_frequency_thz"),
    )

    return channels

def validate_channel_map(channels: Dict[int, ChannelInfo]) -> None:
    if not channels:
        raise ValueError("No channel sections/rows found in map file.")

    used_pixels: Dict[int, int] = {}

    for channel, info in channels.items():
        if not 0 <= info.integer_pixel < LCD_WIDTH:
            raise ValueError(
                f"Channel {channel} maps to invalid pixel {info.integer_pixel};
expected 0..{LCD_WIDTH - 1}."
            )

        if info.integer_pixel in used_pixels:
            other_channel = used_pixels[info.integer_pixel]
            raise ValueError(
                f"Duplicate LCD x pixel {info.integer_pixel}: channel {other_channel}
and channel {channel}."
            )

        used_pixels[info.integer_pixel] = channel

def generate_random_selection(
    channels: Dict[int, ChannelInfo],
    active_count: Optional[int],
    active_probability: Optional[float],
    amp_min: int,
    amp_max: int,
    seed: Optional[int],
) -> Dict[int, int]:
    """
    Return a random Input 2 selection as:
        {channel: amplitude_level}

    Use either active_count or active_probability.
    If neither is supplied, default to roughly 25 percent of channels.
    """
    rng = random.Random(seed)

```

```

available = sorted(channels.keys())

amp_min = clamp_int(amp_min, 0, LCD_HEIGHT)
amp_max = clamp_int(amp_max, 0, LCD_HEIGHT)

if amp_min > amp_max:
    raise ValueError("--amp-min cannot be greater than --amp-max")

if active_count is not None and active_probability is not None:
    raise ValueError("Use either --active-count or --active-probability, not both.")

selected: Dict[int, int] = {}

if active_count is None and active_probability is None:
    active_count = max(1, len(available) // 4)

if active_count is not None:
    active_count = clamp_int(active_count, 0, len(available))
    chosen = rng.sample(available, active_count)

    for channel in chosen:
        selected[channel] = rng.randint(amp_min, amp_max)

    return selected

assert active_probability is not None

if not 0.0 <= active_probability <= 1.0:
    raise ValueError("--active-probability must be between 0.0 and 1.0")

for channel in available:
    if rng.random() <= active_probability:
        selected[channel] = rng.randint(amp_min, amp_max)

return selected

def amplitude_to_y_indices(amplitude: int, vertical_mode: str) -> List[int]:
    amplitude = clamp_int(amplitude, 0, LCD_HEIGHT)

    if amplitude == 0:
        return []

    if vertical_mode == "full":
        return list(range(LCD_HEIGHT))

    if vertical_mode == "bottom":

```



```

        return list(range(LCD_HEIGHT - amplitude, LCD_HEIGHT))

    if vertical_mode == "top":
        return list(range(0, amplitude))

    if vertical_mode == "center":
        start = (LCD_HEIGHT - amplitude) // 2
        return list(range(start, start + amplitude))

    raise ValueError("vertical_mode must be bottom, top, center, or full")

def build_pattern(
    channels: Dict[int, ChannelInfo],
    selected: Dict[int, int],
    vertical_mode: str,
) -> List[List[int]]:
    pattern = [[0 for _ in range(LCD_WIDTH)] for _ in range(LCD_HEIGHT)]

    for channel, amplitude in selected.items():
        info = channels[channel]
        x = info.integer_pixel

        for y in amplitude_to_y_indices(amplitude, vertical_mode):
            pattern[y][x] = 1

    return pattern

def save_txt(pattern: List[List[int]], path: Path) -> None:
    with path.open("w", encoding="utf-8") as file:
        for row in pattern:
            file.write("".join("#" if cell else "." for cell in row) + "\n")

def save_pbm(pattern: List[List[int]], path: Path) -> None:
    with path.open("w", encoding="ascii") as file:
        file.write("P1\n")
        file.write(f"{LCD_WIDTH} {LCD_HEIGHT}\n")
        for row in pattern:
            file.write(" ".join(str(cell) for cell in row) + "\n")

def save_json(
    pattern: List[List[int]],
    channels: Dict[int, ChannelInfo],
    selected: Dict[int, int],
    path: Path,

```

```

    seed: Optional[int],
    vertical_mode: str,
) -> None:
    active = []

    for channel in sorted(selected):
        info = channels[channel]
        active.append({
            "channel": channel,
            "amplitude": selected[channel],
            "x_pixel": info.integer_pixel,
            "prime": info.prime,
            "log2_prime": info.log2_prime,
            "actual_pixel_wavelength_nm": info.actual_pixel_wavelength_nm,
            "actual_pixel_frequency_thz": info.actual_pixel_frequency_thz,
        })

    data = {
        "name": "random_input2_pattern",
        "lcd_width": LCD_WIDTH,
        "lcd_height": LCD_HEIGHT,
        "seed": seed,
        "vertical_mode": vertical_mode,
        "convention": "pattern[y][x], 1=open/pass light, 0=closed/block light",
        "active_channels": active,
        "pattern": pattern,
    }

    with path.open("w", encoding="utf-8") as file:
        json.dump(data, file, indent=2)

def save_c_array(pattern: List[List[int]], path: Path, array_name: str =
"input2_pattern") -> None:
    """
    Save SSD1306-style page bytes:
    page 0 = y 0..7, x 0..127
    page 1 = y 8..15, x 0..127
    ...
    """
    bytes_out: List[int] = []

    for page in range(LCD_HEIGHT // 8):
        for x in range(LCD_WIDTH):
            value = 0
            for bit in range(8):
                y = page * 8 + bit
                if pattern[y][x]:

```

```

        value |= 1 << bit
        bytes_out.append(value)

with path.open("w", encoding="utf-8") as file:
    file.write("#include <stdint.h>\n\n")
    file.write(f"const uint8_t {array_name}[{len(bytes_out)}] = {{\n")

    for i in range(0, len(bytes_out), 16):
        chunk = bytes_out[i:i + 16]
        file.write("    " + ", ".join(f"0x{byte:02X}" for byte in chunk))
        if i + 16 < len(bytes_out):
            file.write(",")
        file.write("\n")

    file.write("};\n")

def save_outputs(
    pattern: List[List[int]],
    channels: Dict[int, ChannelInfo],
    selected: Dict[int, int],
    out_prefix: str,
    seed: Optional[int],
    vertical_mode: str,
) -> None:
    base = Path(out_prefix)

    save_txt(pattern, base.with_suffix(".txt"))
    save_pbm(pattern, base.with_suffix(".pbm"))
    save_json(pattern, channels, selected, base.with_suffix(".json"), seed,
vertical_mode)
    save_c_array(pattern, base.with_suffix(".c"))

    print(f"Saved {base.with_suffix('.txt')}")
    print(f"Saved {base.with_suffix('.pbm')}")
    print(f"Saved {base.with_suffix('.json')}")
    print(f"Saved {base.with_suffix('.c')}")

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="Standalone random Input 2 LCD
pattern generator.")

    parser.add_argument("--map", required=True, help="Input wavelength map .cfg or
.csv")
    parser.add_argument("--out", default="input2_random", help="Output file prefix")

    group = parser.add_mutually_exclusive_group()

```

```

    group.add_argument("--active-count", type=int, default=None, help="Exact number of
channels to activate")
    group.add_argument("--active-probability", type=float, default=None, help="Chance
each channel is active, 0.0 to 1.0")

    parser.add_argument("--amp-min", type=int, default=1, help="Minimum amplitude
level, 0..64")
    parser.add_argument("--amp-max", type=int, default=LCD_HEIGHT, help="Maximum
amplitude level, 0..64")
    parser.add_argument("--seed", type=int, default=None, help="Seed for repeatable
random patterns")
    parser.add_argument(
        "--vertical-mode",
        choices=["bottom", "top", "center", "full"],
        default="bottom",
        help="How amplitude maps onto the 64-pixel vertical axis"
    )

    return parser

def main() -> None:
    args = build_parser().parse_args()

    channels = load_channel_map(args.map)

    selected = generate_random_selection(
        channels=channels,
        active_count=args.active_count,
        active_probability=args.active_probability,
        amp_min=args.amp_min,
        amp_max=args.amp_max,
        seed=args.seed,
    )

    pattern = build_pattern(
        channels=channels,
        selected=selected,
        vertical_mode=args.vertical_mode,
    )

    save_outputs(
        pattern=pattern,
        channels=channels,
        selected=selected,
        out_prefix=args.out,
        seed=args.seed,
        vertical_mode=args.vertical_mode,

```

```

)

print()
print("Random Input 2 Summary")
print("-----")
print(f"Mapped channels available: {len(channels)}")
print(f"Active channels selected: {len(selected)}")
print(f"Seed: {args.seed}")
print(f"Amplitude range: {args.amp_min}..{args.amp_max}")
print(f"Vertical mode: {args.vertical_mode}")
print()

for channel in sorted(selected):
    info = channels[channel]
    print(
        f"channel={channel:>3}  "
        f"amp={selected[channel]:>2}  "
        f"x={info.integer_pixel:>3}  "
        f"prime={info.prime}  "
        f"wavelength_nm={info.actual_pixel_wavelength_nm}"
    )

if __name__ == "__main__":
    main()

```

Optical_LCD_Codec.py

```

#!/usr/bin/env python3
"""
random_input2_generator.py

Standalone random Input 2 pattern generator for the optical LCD core.

Purpose:
- Load your log-prime wavelength map from .cfg or .csv.
- Randomly choose channels for Input 2.
- Generate a 128x64 LCD pattern.
- Save the pattern as:
    input2_random.json  machine-readable
    input2_random.txt   human-readable
    input2_random.pbm   monochrome PBM preview
    input2_random.c     SSD1306-style byte array

Conventions:

```

- x = 0..127 = wavelength column axis.
- y = 0..63 = amplitude axis.
- 1 = open / pass light.
- 0 = closed / block light.

Examples:

```
python random_input2_generator.py --map integer_log_prime_wavelength_map.cfg --active-count 8 --out input2_random
```

```
python random_input2_generator.py --map integer_log_prime_wavelength_map.csv --active-probability 0.25 --seed 1234 --out input2_seeded
```

```
python random_input2_generator.py --map map.cfg --active-count 12 --amp-min 8 --amp-max 64 --vertical-mode bottom
```

```
import argparse
import configparser
import csv
import json
import random
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional

LCD_WIDTH = 128
LCD_HEIGHT = 1

@dataclass(frozen=True)
class ChannelInfo:
    channel: int
    prime: int
    log2_prime: float
    integer_pixel: int
    actual_pixel_wavelength_nm: Optional[float] = None
    actual_pixel_frequency_thz: Optional[float] = None
```

```
def clamp_int(value: int, low: int, high: int) -> int:
    return max(low, min(high, value))
```

```
def clean_header(header: str) -> str:
    """
    Makes CSV loading tolerant of pasted rich-text headers.
    """
    if header is None:
        return ""
```

```

text = str(header).strip()
text = text.replace('<strong data-lexical-text="true">', '')
text = text.replace('</strong>', '')
text = text.replace(',', '')
return text.strip()

def optional_float(mapping, key: str) -> Optional[float]:
    if key not in mapping:
        return None

    value = mapping[key]

    if value is None:
        return None

    value = str(value).strip()

    if value == "":
        return None

    return float(value)

def load_channel_map(path: str | Path) -> Dict[int, ChannelInfo]:
    path = Path(path)

    if not path.exists():
        raise FileNotFoundError(f"Map file not found: {path}")

    if path.suffix.lower() == ".cfg":
        channels = load_channel_map_cfg(path)
    elif path.suffix.lower() == ".csv":
        channels = load_channel_map_csv(path)
    else:
        raise ValueError("Map file must be .cfg or .csv")

    validate_channel_map(channels)
    return channels

def load_channel_map_cfg(path: Path) -> Dict[int, ChannelInfo]:
    cfg = configparser.ConfigParser()
    cfg.read(path, encoding="utf-8")

    channels: Dict[int, ChannelInfo] = {}

    for section in cfg.sections():

```

```

    if not section.lower().startswith("channel_"):
        continue

    item = cfg[section]

    channel = int(item["channel"])
    prime = int(item["prime"])
    log2_prime = float(item["log2_prime"])
    integer_pixel = int(item["integer_pixel"])

    channels[channel] = ChannelInfo(
        channel=channel,
        prime=prime,
        log2_prime=log2_prime,
        integer_pixel=integer_pixel,
        actual_pixel_wavelength_nm=optional_float(item,
"actual_pixel_wavelength_nm"),
        actual_pixel_frequency_thz=optional_float(item,
"actual_pixel_frequency_thz"),
    )

    return channels

def load_channel_map_csv(path: Path) -> Dict[int, ChannelInfo]:
    channels: Dict[int, ChannelInfo] = {}

    with path.open("r", newline="", encoding="utf-8-sig") as file:
        reader = csv.DictReader(file)

        for row in reader:
            clean = {clean_header(key): value for key, value in row.items()}

            channel = int(clean["channel"])
            prime = int(clean["prime"])
            log2_prime = float(clean["log2_prime"])
            integer_pixel = int(clean["integer_pixel"])

            channels[channel] = ChannelInfo(
                channel=channel,
                prime=prime,
                log2_prime=log2_prime,
                integer_pixel=integer_pixel,
                actual_pixel_wavelength_nm=optional_float(clean,
"actual_pixel_wavelength_nm"),
                actual_pixel_frequency_thz=optional_float(clean,
"actual_pixel_frequency_thz"),
            )

```



```

return channels

def validate_channel_map(channels: Dict[int, ChannelInfo]) -> None:
    if not channels:
        raise ValueError("No channel sections/rows found in map file.")

    used_pixels: Dict[int, int] = {}

    for channel, info in channels.items():
        if not 0 <= info.integer_pixel < LCD_WIDTH:
            raise ValueError(
                f"Channel {channel} maps to invalid pixel {info.integer_pixel};
expected 0..{LCD_WIDTH - 1}."
            )

            if info.integer_pixel in used_pixels:
                other_channel = used_pixels[info.integer_pixel]
                raise ValueError(
                    f"Duplicate LCD x pixel {info.integer_pixel}: channel {other_channel}
and channel {channel}."
                )

            used_pixels[info.integer_pixel] = channel

def generate_random_selection(
    channels: Dict[int, ChannelInfo],
    active_count: Optional[int],
    active_probability: Optional[float],
    amp_min: int,
    amp_max: int,
    seed: Optional[int],
) -> Dict[int, int]:
    """
    Return a random Input 2 selection as:
        {channel: amplitude_level}

    Use either active_count or active_probability.
    If neither is supplied, default to roughly 25 percent of channels.
    """
    rng = random.Random(seed)
    available = sorted(channels.keys())

    amp_min = clamp_int(amp_min, 0, LCD_HEIGHT)
    amp_max = clamp_int(amp_max, 0, LCD_HEIGHT)

```

```

if amp_min > amp_max:
    raise ValueError("--amp-min cannot be greater than --amp-max")

if active_count is not None and active_probability is not None:
    raise ValueError("Use either --active-count or --active-probability, not
both.")

selected: Dict[int, int] = {}

if active_count is None and active_probability is None:
    active_count = max(1, len(available) // 4)

if active_count is not None:
    active_count = clamp_int(active_count, 0, len(available))
    chosen = rng.sample(available, active_count)

    for channel in chosen:
        selected[channel] = rng.randint(amp_min, amp_max)

    return selected

assert active_probability is not None

if not 0.0 <= active_probability <= 1.0:
    raise ValueError("--active-probability must be between 0.0 and 1.0")

for channel in available:
    if rng.random() <= active_probability:
        selected[channel] = rng.randint(amp_min, amp_max)

return selected

def amplitude_to_y_indices(amplitude: int, vertical_mode: str) -> List[int]:
    amplitude = clamp_int(amplitude, 0, LCD_HEIGHT)

    if amplitude == 0:
        return []

    if vertical_mode == "full":
        return list(range(LCD_HEIGHT))

    if vertical_mode == "bottom":
        return list(range(LCD_HEIGHT - amplitude, LCD_HEIGHT))

    if vertical_mode == "top":
        return list(range(0, amplitude))

```

```

    if vertical_mode == "center":
        start = (LCD_HEIGHT - amplitude) // 2
        return list(range(start, start + amplitude))

    raise ValueError("vertical_mode must be bottom, top, center, or full")

def build_pattern(
    channels: Dict[int, ChannelInfo],
    selected: Dict[int, int],
    vertical_mode: str,
) -> List[List[int]]:
    pattern = [[0 for _ in range(LCD_WIDTH)] for _ in range(LCD_HEIGHT)]

    for channel, amplitude in selected.items():
        info = channels[channel]
        x = info.integer_pixel

        for y in amplitude_to_y_indices(amplitude, vertical_mode):
            pattern[y][x] = 1

    return pattern

def save_txt(pattern: List[List[int]], path: Path) -> None:
    with path.open("w", encoding="utf-8") as file:
        for row in pattern:
            file.write("".join("#" if cell else "." for cell in row) + "\n")

def save_pbm(pattern: List[List[int]], path: Path) -> None:
    with path.open("w", encoding="ascii") as file:
        file.write("P1\n")
        file.write(f"{LCD_WIDTH} {LCD_HEIGHT}\n")
        for row in pattern:
            file.write(" ".join(str(cell) for cell in row) + "\n")

def save_json(
    pattern: List[List[int]],
    channels: Dict[int, ChannelInfo],
    selected: Dict[int, int],
    path: Path,
    seed: Optional[int],
    vertical_mode: str,
) -> None:
    active = []

```

```

for channel in sorted(selected):
    info = channels[channel]
    active.append({
        "channel": channel,
        "amplitude": selected[channel],
        "x_pixel": info.integer_pixel,
        "prime": info.prime,
        "log2_prime": info.log2_prime,
        "actual_pixel_wavelength_nm": info.actual_pixel_wavelength_nm,
        "actual_pixel_frequency_thz": info.actual_pixel_frequency_thz,
    })

data = {
    "name": "random_input2_pattern",
    "lcd_width": LCD_WIDTH,
    "lcd_height": LCD_HEIGHT,
    "seed": seed,
    "vertical_mode": vertical_mode,
    "convention": "pattern[y][x], 1=open/pass light, 0=closed/block light",
    "active_channels": active,
    "pattern": pattern,
}

with path.open("w", encoding="utf-8") as file:
    json.dump(data, file, indent=2)

def save_c_array(pattern: List[List[int]], path: Path, array_name: str =
"input2_pattern") -> None:
    """
    Save SSD1306-style page bytes:
    page 0 = y 0..7, x 0..127
    page 1 = y 8..15, x 0..127
    ...
    """
    bytes_out: List[int] = []

    for page in range(LCD_HEIGHT // 8):
        for x in range(LCD_WIDTH):
            value = 0
            for bit in range(8):
                y = page * 8 + bit
                if pattern[y][x]:
                    value |= 1 << bit
            bytes_out.append(value)

    with path.open("w", encoding="utf-8") as file:
        file.write("#include <stdint.h>\n\n")

```

```

        file.write(f"const uint8_t {array_name}[{len(bytes_out)}] = {{\n")

        for i in range(0, len(bytes_out), 16):
            chunk = bytes_out[i:i + 16]
            file.write("    " + ", ".join(f"0x{byte:02X}" for byte in chunk))
            if i + 16 < len(bytes_out):
                file.write(",")
            file.write("\n")

        file.write("};\n")

def save_outputs(
    pattern: List[List[int]],
    channels: Dict[int, ChannelInfo],
    selected: Dict[int, int],
    out_prefix: str,
    seed: Optional[int],
    vertical_mode: str,
) -> None:
    base = Path(out_prefix)

    save_txt(pattern, base.with_suffix(".txt"))
    save_pbm(pattern, base.with_suffix(".pbm"))
    save_json(pattern, channels, selected, base.with_suffix(".json"), seed,
vertical_mode)
    save_c_array(pattern, base.with_suffix(".c"))

    print(f"Saved {base.with_suffix('.txt')}")
    print(f"Saved {base.with_suffix('.pbm')}")
    print(f"Saved {base.with_suffix('.json')}")
    print(f"Saved {base.with_suffix('.c')}")

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="Standalone random Input 2 LCD
pattern generator.")

    parser.add_argument("--map", required=True, help="Input wavelength map .cfg or
.csv")
    parser.add_argument("--out", default="input2_random", help="Output file prefix")

    group = parser.add_mutually_exclusive_group()
    group.add_argument("--active-count", type=int, default=None, help="Exact number of
channels to activate")
    group.add_argument("--active-probability", type=float, default=None, help="Chance
each channel is active, 0.0 to 1.0")

```

```

    parser.add_argument("--amp-min", type=int, default=1, help="Minimum amplitude
level, 0..64")
    parser.add_argument("--amp-max", type=int, default=LCD_HEIGHT, help="Maximum
amplitude level, 0..64")
    parser.add_argument("--seed", type=int, default=None, help="Seed for repeatable
random patterns")
    parser.add_argument(
        "--vertical-mode",
        choices=["bottom", "top", "center", "full"],
        default="bottom",
        help="How amplitude maps onto the 64-pixel vertical axis"
    )

    return parser

def main() -> None:
    args = build_parser().parse_args()

    channels = load_channel_map(args.map)

    selected = generate_random_selection(
        channels=channels,
        active_count=args.active_count,
        active_probability=args.active_probability,
        amp_min=args.amp_min,
        amp_max=args.amp_max,
        seed=args.seed,
    )

    pattern = build_pattern(
        channels=channels,
        selected=selected,
        vertical_mode=args.vertical_mode,
    )

    save_outputs(
        pattern=pattern,
        channels=channels,
        selected=selected,
        out_prefix=args.out,
        seed=args.seed,
        vertical_mode=args.vertical_mode,
    )

    print()
    print("Random Input 2 Summary")
    print("-----")

```

```

print(f"Mapped channels available: {len(channels)}")
print(f"Active channels selected: {len(selected)}")
print(f"Seed: {args.seed}")
print(f"Amplitude range: {args.amp_min}..{args.amp_max}")
print(f"Vertical mode: {args.vertical_mode}")
print()

for channel in sorted(selected):
    info = channels[channel]
    print(
        f"channel={channel:>3}  "
        f"amp={selected[channel]:>2}  "
        f"x={info.integer_pixel:>3}  "
        f"prime={info.prime}  "
        f"wavelength_nm={info.actual_pixel_wavelength_nm}"
    )

if __name__ == "__main__":
    main()

```

Nor.py

```

#!/usr/bin/env python3
"""
nor.py

Standalone 128-column NOR combiner for debug and file generation.

Purpose:
- Read Input 1 and Input 2 as 128 numbers each.
- Treat any value > 0 as logic 1 / active / light present.
- Compute lane-wise NOR:

    OUT[i] = NOT(INPUT1[i] OR INPUT2[i])

which means:

    OUT[i] = 1 only when INPUT1[i] == 0 and INPUT2[i] == 0
    OUT[i] = 0 otherwise

- Save the output as exactly 128 numbers.

Important:
- This script operates across all 128 LCD columns.

```

- It does NOT know which columns are real mapped optical wavelength lanes.
- main.py performs the physical mapped-lane NOR by masking unmapped columns.
- Therefore this script should not calculate optical voltage.

Supported input formats:

- .txt / .csv containing 128 numbers separated by spaces, commas, or newlines
- .json containing either:


```
[0, 1, 0, ...]
{"values": [0, 1, 0, ...]}
{"output": [0, 1, 0, ...]}
{"pattern": [...128...], ...}]
```

For 2D pattern JSON:

- Each x-column becomes one number.
- If any pixel in the column is > 0, that output lane is 1.

Output files:

- <out>.txt
- <out>.csv
- <out>.json
- <out>.c

Examples:

```
py -3.13 nor.py --input1 input1.json --input2 input2.json --out nor_result
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import csv
```

```
import json
```

```
import re
```

```
from pathlib import Path
```

```
from typing import Any, List
```

```
LANE_COUNT = 128
```

```
def load_128_values(path: str | Path, active_threshold: float = 0.0) -> List[int]:
    """
```

Load exactly 128 logical lane values from a file.

Returned values are binary:

0 = inactive / no light / false

1 = active / light present / true

Any numeric value greater than active_threshold becomes 1.

Any numeric value <= active_threshold becomes 0.


```

"""
path = Path(path)

if not path.exists():
    raise FileNotFoundError(f"Input file not found: {path}")

suffix = path.suffix.lower()

if suffix == ".json":
    raw_values = load_values_from_json(path)
else:
    raw_values = load_values_from_text(path)

if len(raw_values) != LANE_COUNT:
    raise ValueError(
        f"{path} produced {len(raw_values)} values; expected exactly
{LANE_COUNT}."
    )

return [1 if float(value) > active_threshold else 0 for value in raw_values]

def load_values_from_json(path: Path) -> List[float]:
    with path.open("r", encoding="utf-8") as file:
        data = json.load(file)

    if isinstance(data, list):
        return flatten_direct_list(data)

    if not isinstance(data, dict):
        raise ValueError(f"Unsupported JSON structure in {path}")

    for key in ["values", "output", "lanes", "input", "input1", "input2"]:
        if key in data:
            return flatten_direct_list(data[key])

    if "pattern" in data:
        return compress_2d_pattern_to_128(data["pattern"])

    raise ValueError(
        f"JSON file {path} must contain a direct list, a 'values'/'output' list, or a
'pattern' matrix."
    )

def flatten_direct_list(value: Any) -> List[float]:
    if not isinstance(value, list):
        raise ValueError("Expected a list of numeric values.")

```

```

    if len(value) == LANE_COUNT and all(not isinstance(item, list) for item in value):
        return [float(item) for item in value]

    if value and all(isinstance(row, list) for row in value):
        return compress_2d_pattern_to_128(value)

    raise ValueError("Expected either a 128-number list or a 2D pattern list.")

def compress_2d_pattern_to_128(pattern: List[List[Any]]) -> List[float]:
    """
    Convert a 2D LCD pattern to 128 lane values.

    Any open pixel in a column means the column/lane is active.
    """
    if not pattern:
        raise ValueError("Pattern is empty.")

    width = len(pattern[0])

    if width != LANE_COUNT:
        raise ValueError(f"Pattern width is {width}; expected {LANE_COUNT}.")

    for y, row in enumerate(pattern):
        if len(row) != LANE_COUNT:
            raise ValueError(f"Pattern row {y} has width {len(row)}; expected {LANE_COUNT}.")

    output: List[float] = []

    for x in range(LANE_COUNT):
        active = any(float(pattern[y][x]) > 0 for y in range(len(pattern)))
        output.append(1.0 if active else 0.0)

    return output

def load_values_from_text(path: Path) -> List[float]:
    text = path.read_text(encoding="utf-8").strip()

    if not text:
        raise ValueError(f"Input file is empty: {path}")

    if text.startswith("P1"):
        return load_values_from_pbm_text(text)

    tokens = re.split(r"[\s,;]+", text)

```

```

tokens = [token for token in tokens if token != ""]

return [float(token) for token in tokens]

def load_values_from_pbm_text(text: str) -> List[float]:
    """
    Supports P1 PBM.

    If 128x1, returns the row directly.
    If 128x64 or other 128-wide PBM, compresses by column.
    """
    lines: List[str] = []

    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        lines.append(line)

    if not lines or lines[0] != "P1":
        raise ValueError("Invalid P1 PBM file.")

    if len(lines) < 3:
        raise ValueError("PBM file is missing dimensions or pixel data.")

    width, height = [int(part) for part in lines[1].split()[:2]]

    if width != LANE_COUNT:
        raise ValueError(f"PBM width is {width}; expected {LANE_COUNT}.")

    tokens: List[str] = []
    for line in lines[2:]:
        tokens.extend(line.split())

    values = [float(token) for token in tokens]

    if len(values) != width * height:
        raise ValueError(
            f"PBM has {len(values)} pixels; expected {width * height}."
        )

    if height == 1:
        return values

    pattern: List[List[float]] = []

    for y in range(height):

```

```

        start = y * width
        pattern.append(values[start:start + width])

    return compress_2d_pattern_to_128(pattern)

def nor_128(input1: List[int], input2: List[int]) -> List[int]:
    if len(input1) != LANE_COUNT or len(input2) != LANE_COUNT:
        raise ValueError("Both inputs must contain exactly 128 values.")

    return [
        1 if input1[i] == 0 and input2[i] == 0 else 0
        for i in range(LANE_COUNT)
    ]

def save_txt(values: List[int], path: Path) -> None:
    path.write_text(
        " ".join(str(value) for value in values) + "\n",
        encoding="utf-8"
    )

def save_csv(values: List[int], path: Path) -> None:
    with path.open("w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(values)

def save_json(input1: List[int], input2: List[int], output: List[int], path: Path) ->
None:
    data = {
        "lane_count": LANE_COUNT,
        "logic": "OUT[i] = NOT(INPUT1[i] OR INPUT2[i])",
        "convention": "0=inactive/no-light/false, 1=active/light-present/true",
        "input1": input1,
        "input2": input2,
        "output": output,
        "active_output_indices": [
            i for i, value in enumerate(output)
            if value == 1
        ],
    }

    with path.open("w", encoding="utf-8") as file:
        json.dump(data, file, indent=2)

```

```

def save_c_array(values: List[int], path: Path, array_name: str = "nor_output") ->
None:
    with path.open("w", encoding="utf-8") as file:
        file.write("#include <stdint.h>\n\n")
        file.write(f"const uint8_t {array_name}[{LANE_COUNT}] = {{\n")

        for i in range(0, LANE_COUNT, 16):
            chunk = values[i:i + 16]
            file.write("    " + ", ".join(str(value) for value in chunk))

            if i + 16 < LANE_COUNT:
                file.write(",")

            file.write("\n")

        file.write("};\n")

def save_outputs(
    input1: List[int],
    input2: List[int],
    output: List[int],
    out_prefix: str
) -> None:
    base = Path(out_prefix)

    save_txt(output, base.with_suffix(".txt"))
    save_csv(output, base.with_suffix(".csv"))
    save_json(input1, input2, output, base.with_suffix(".json"))
    save_c_array(output, base.with_suffix(".c"))

    print(f"Saved {base.with_suffix('.txt')}")
    print(f"Saved {base.with_suffix('.csv')}")
    print(f"Saved {base.with_suffix('.json')}")
    print(f"Saved {base.with_suffix('.c')}")

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="NOR two 128-column LCD/vector outputs. Debug helper only."
    )

    parser.add_argument("--input1", required=True, help="First 128-number input file")
    parser.add_argument("--input2", required=True, help="Second 128-number input
file")
    parser.add_argument("--out", default="nor_result", help="Output file prefix")
    parser.add_argument(
        "--active-threshold",

```

```

        type=float,
        default=0.0,
        help="Values greater than this are treated as logic 1. Default: 0.0"
    )

    return parser

def main() -> None:
    args = build_parser().parse_args()

    input1 = load_128_values(args.input1, active_threshold=args.active_threshold)
    input2 = load_128_values(args.input2, active_threshold=args.active_threshold)

    output = nor_128(input1, input2)

    save_outputs(input1, input2, output, args.out)

    print()
    print("128-Column NOR Debug Summary")
    print("-----")
    print(f"Input 1 active columns: {sum(input1)}")
    print(f"Input 2 active columns: {sum(input2)}")
    print(f"NOR active columns: {sum(output)}")
    print(f"Active indices:      [{i for i, value in enumerate(output) if value == 1}]")
    print()
    print("Note:")
    print("  This is a raw 128-column LCD/vector NOR debug result.")
    print("  It is not the physical mapped optical NOR result.")
    print("  main.py masks this down to the mapped wavelength lanes.")
    print("  Use main.py output for the actual optical-core result.")

if __name__ == "__main__":
    main()

```

Nor_voltage_decoder.py

```

#!/usr/bin/env python3
"""
nor_voltage_decoder.py

Fast unknown mean-voltage decoder for the log-prime optical NOR system.

```

This is the replacement decoder for the current Optical GPU Driver prototype.

Core model

The optical output voltage is treated as the mean log-prime channel voltage:

$$V_{\text{measured}} = V_{\text{OFFSET}} + K * \text{mean}(\ln(p_i) \text{ for active mapped channels})$$

So:

$$L = (V_{\text{measured}} - V_{\text{OFFSET}}) / K$$

For an unknown active count k:

$$k * L = \text{sum}(\ln(p_i)) = \ln(\text{product}(p_i))$$

Therefore:

$$\text{product_guess} = \text{round}(\exp(k * L))$$

Instead of enumerating all subsets, this decoder uses the known prime mapping. It tries possible active counts, reconstructs the candidate product, then extracts which mapped primes are present using a product tree and grouped GCD searches.

Why product tree?

The simple decoder checks every prime for every active-count guess.

This version groups primes into a binary product tree. For each subtree:

$$\text{shared} = \text{gcd}(\text{target_product}, \text{subtree_product})$$

If shared == 1:

no active prime in that subtree

If shared == subtree_product:

the entire subtree is active

Otherwise:

recurse into the subtree

This avoids brute-force subset enumeration and avoids naive trial division over every prime in the common cases.

Typical usage

Decode from actual NOR vector by simulating measured voltage:

```
py -3.13 nor_voltage_decoder.py --map integer_log_prime_wavelength_map.cfg --
actual-nor nor_result.json --simulate-from-actual
```

Decode from a real measured voltage and compare against the actual vector:

```
py -3.13 nor_voltage_decoder.py --map integer_log_prime_wavelength_map.cfg --
measured-voltage 1.543685 --actual-nor nor_result.json
```

Quiet run:

```
py -3.13 nor_voltage_decoder.py --map integer_log_prime_wavelength_map.cfg --
actual-nor nor_result.json --simulate-from-actual --quiet
```

```
"""
```

```
from __future__ import annotations

import argparse
import configparser
import csv
import json
import math
import re
import time
from dataclasses import dataclass
from decimal import Decimal, ROUND_HALF_EVEN, getcontext
from pathlib import Path
from typing import Any, Dict, Iterable, List, Optional, Tuple
```

```
LANE_COUNT = 128
DEFAULT_V_OFFSET = Decimal("1.0")
DEFAULT_K = Decimal("0.1")
```

```
@dataclass(slots=True)
class Channel:
    channel: int
    prime: int
    pixel: int
    ln_prime: Optional[Decimal] = None
```

```
@dataclass(slots=True)
class ProductTreeNode:
    product: int
    channels: List[Channel]
    left: Optional["ProductTreeNode"] = None
    right: Optional["ProductTreeNode"] = None
```



```

@property
def is_leaf(self) -> bool:
    return self.left is None and self.right is None

# -----
# Formatting helpers
# -----

def short_decimal(value: Decimal, significant_digits: int = 12) -> str:
    """Return a compact scientific-notation string for huge Decimals."""
    if value.is_zero():
        return "0"

    sign = "-" if value < 0 else ""
    value = abs(value)
    exponent = value.adjusted()
    mantissa = value.scaleb(-exponent)
    return f"{sign}{mantissa:.{significant_digits - 1}f}e{exponent}"

def decimal_to_json(value: Decimal) -> str:
    return str(value)

# -----
# Map loading
# -----

def clean_header(header: str) -> str:
    """Make CSV loading tolerant of pasted rich-text headers."""
    if header is None:
        return ""

    text = str(header).strip()
    text = text.replace('<strong data-lexical-text="true">', '')
    text = text.replace('</strong>', '')
    text = text.replace(',', '')
    return text.strip()

def load_map(path: str | Path) -> List[Channel]:
    path = Path(path)

    if not path.exists():
        raise FileNotFoundError(f"Map file not found: {path}")

    if path.suffix.lower() == ".cfg":

```

```

        channels = load_map_cfg(path)
    elif path.suffix.lower() == ".csv":
        channels = load_map_csv(path)
    else:
        raise ValueError("Map must be .cfg or .csv")

    return validate_channels(channels)

def load_map_cfg(path: Path) -> List[Channel]:
    cfg = configparser.ConfigParser()
    cfg.read(path, encoding="utf-8")

    channels: List[Channel] = []

    for section in cfg.sections():
        if not section.lower().startswith("channel_"):
            continue

        item = cfg[section]
        channels.append(
            Channel(
                channel=int(item["channel"]),
                prime=int(item["prime"]),
                pixel=int(item["integer_pixel"]),
            )
        )

    return channels

def load_map_csv(path: Path) -> List[Channel]:
    channels: List[Channel] = []

    with path.open("r", newline="", encoding="utf-8-sig") as file:
        reader = csv.DictReader(file)

        for row in reader:
            clean = {clean_header(key): value for key, value in row.items()}
            channels.append(
                Channel(
                    channel=int(clean["channel"]),
                    prime=int(clean["prime"]),
                    pixel=int(clean["integer_pixel"]),
                )
            )

    return channels

```

```

def validate_channels(channels: List[Channel]) -> List[Channel]:
    if not channels:
        raise ValueError("No channels found in map.")

    seen_pixels: Dict[int, int] = {}
    seen_primes: Dict[int, int] = {}

    for channel in channels:
        if not 0 <= channel.pixel < LANE_COUNT:
            raise ValueError(f"Channel {channel.channel} has invalid pixel {channel.pixel}.")

        if channel.pixel in seen_pixels:
            raise ValueError(
                f"Duplicate pixel {channel.pixel}: "
                f"channel {seen_pixels[channel.pixel]} and {channel.channel}."
            )

        if channel.prime in seen_primes:
            raise ValueError(
                f"Duplicate prime {channel.prime}: "
                f"channel {seen_primes[channel.prime]} and {channel.channel}."
            )

        seen_pixels[channel.pixel] = channel.channel
        seen_primes[channel.prime] = channel.channel

    # Keep logical channel order stable.
    return sorted(channels, key=lambda item: item.channel)

def precompute_logs(channels: List[Channel]) -> None:
    for channel in channels:
        channel.ln_prime = Decimal(channel.prime).ln()

# -----
# 128 vector loading
# -----

def load_128_vector(path: str | Path) -> List[int]:
    path = Path(path)

    if not path.exists():
        raise FileNotFoundError(f"Vector file not found: {path}")

```

```

if path.suffix.lower() == ".json":
    data = json.loads(path.read_text(encoding="utf-8"))

    if isinstance(data, list):
        return normalize_list_or_pattern(data)

    if isinstance(data, dict):
        for key in ["output", "values", "lanes", "input", "input1", "input2"]:
            if key in data:
                return normalize_list_or_pattern(data[key])

        if "pattern" in data:
            return compress_pattern_to_vector(data["pattern"])

        raise ValueError("JSON must contain output/values/pattern")

text = path.read_text(encoding="utf-8").strip()

if text.startswith("P1"):
    return load_pbm_vector(text)

tokens = [token for token in re.split(r"[\s,;]+", text) if token]
values = [1 if Decimal(token) > 0 else 0 for token in tokens]

if len(values) != LANE_COUNT:
    raise ValueError(f"Expected {LANE_COUNT} values, got {len(values)} from {path}")

return values

def normalize_list_or_pattern(value: Any) -> List[int]:
    if not isinstance(value, list):
        raise ValueError("Expected list")

    if len(value) == LANE_COUNT and all(not isinstance(item, list) for item in value):
        return [1 if Decimal(str(item)) > 0 else 0 for item in value]

    if value and all(isinstance(row, list) for row in value):
        return compress_pattern_to_vector(value)

    raise ValueError("Expected 128-number list or 2D pattern")

def compress_pattern_to_vector(pattern: List[List[Any]]) -> List[int]:
    if not pattern:
        raise ValueError("Pattern is empty")

```

```

    if len(pattern[0]) != LANE_COUNT:
        raise ValueError(f"Pattern width expected {LANE_COUNT}, got {len(pattern[0])}")

    for row_index, row in enumerate(pattern):
        if len(row) != LANE_COUNT:
            raise ValueError(f"Pattern row {row_index} has wrong width")

    return [
        1 if any(Decimal(str(pattern[y][x])) > 0 for y in range(len(pattern))) else 0
        for x in range(LANE_COUNT)
    ]

def load_pbm_vector(text: str) -> List[int]:
    lines: List[str] = []

    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        lines.append(line)

    if not lines or lines[0] != "P1":
        raise ValueError("Invalid PBM")

    width, height = [int(part) for part in lines[1].split()[:2]]

    if width != LANE_COUNT:
        raise ValueError(f"Expected PBM width {LANE_COUNT}, got {width}")

    tokens: List[str] = []
    for line in lines[2:]:
        tokens.extend(line.split())

    values = [1 if Decimal(token) > 0 else 0 for token in tokens]

    if len(values) != width * height:
        raise ValueError(f"PBM expected {width * height} pixels, got {len(values)}")

    if height == 1:
        return values

    pattern: List[List[int]] = []

    for y in range(height):
        start = y * width
        pattern.append(values[start:start + width])

```

```

    return compress_pattern_to_vector(pattern)

# -----
# Voltage model and vectors
# -----

def mean_voltage_for_channels(active_channels: List[Channel], v_offset: Decimal,
                              k_value: Decimal) -> Decimal:
    if not active_channels:
        return Decimal("0")

    total_log = sum(channel.ln_prime for channel in active_channels if
channel.ln_prime is not None)
    mean_log = total_log / Decimal(len(active_channels))
    return v_offset + k_value * mean_log

def vector_from_active_channels(active_channels: List[Channel]) -> List[int]:
    vector = [0 for _ in range(LANE_COUNT)]

    for channel in active_channels:
        vector[channel.pixel] = 1

    return vector

def active_channels_from_vector(channels: List[Channel], vector: List[int]) ->
List[Channel]:
    pixel_to_channel = {channel.pixel: channel for channel in channels}
    active: List[Channel] = []

    for pixel, value in enumerate(vector):
        if value and pixel in pixel_to_channel:
            active.append(pixel_to_channel[pixel])

    return sorted(active, key=lambda item: item.channel)

# -----
# Product-tree decoding
# -----

def build_product_tree(channels: List[Channel]) -> ProductTreeNode:
    if not channels:
        raise ValueError("Cannot build product tree from no channels.")

```

```

if len(channels) == 1:
    return ProductTreeNode(
        product=channels[0].prime,
        channels=channels,
    )

midpoint = len(channels) // 2
left = build_product_tree(channels[:midpoint])
right = build_product_tree(channels[midpoint:])

return ProductTreeNode(
    product=left.product * right.product,
    channels=channels,
    left=left,
    right=right,
)

def extract_channels_from_product_tree(node: ProductTreeNode, target_product: int) ->
List[Channel]:
    """
    Extract active channels from target_product using grouped gcd recursion.

    This is the non-linear factor recovery step:
    - gcd == 1 means no active factor in this subtree.
    - gcd == subtree product means the whole subtree is active.
    - otherwise recurse into the subtree.
    """
    shared = math.gcd(target_product, node.product)

    if shared == 1:
        return []

    if shared == node.product:
        return node.channels.copy()

    if node.is_leaf:
        return node.channels.copy() if shared == node.product else []

    decoded: List[Channel] = []

    if node.left is not None:
        decoded.extend(extract_channels_from_product_tree(node.left, shared))

    if node.right is not None:
        decoded.extend(extract_channels_from_product_tree(node.right, shared))

    return decoded

```

```

def product_is_known_subset(product_tree: ProductTreeNode, target_product: int) ->
bool:
    if target_product <= 1:
        return False

    shared = math.gcd(target_product, product_tree.product)
    return shared == target_product

# -----
# Active-count pruning
# -----

def build_count_windows(channels: List[Channel]) -> Dict[int, Tuple[Decimal,
Decimal]]:
    """
    For each active count k, compute min and max possible mean ln(prime).

    This prunes impossible active counts before expensive Decimal.exp calls.
    """
    logs = sorted(channel.ln_prime for channel in channels if channel.ln_prime is not
None)
    n = len(logs)

    prefix = [Decimal("0")]

    for value in logs:
        prefix.append(prefix[-1] + value)

    windows: Dict[int, Tuple[Decimal, Decimal]] = {}

    for k in range(1, n + 1):
        min_mean = (prefix[k] - prefix[0]) / Decimal(k)
        max_mean = (prefix[n] - prefix[n - k]) / Decimal(k)
        windows[k] = (min_mean, max_mean)

    return windows

def candidate_counts_for_mean(
    normalized_mean_log: Decimal,
    windows: Dict[int, Tuple[Decimal, Decimal]],
    log_tolerance: Decimal,
) -> List[int]:
    candidates: List[Tuple[Decimal, int]] = []

```



```

    for active_count, (min_mean, max_mean) in windows.items():
        if min_mean - log_tolerance <= normalized_mean_log <= max_mean +
log_tolerance:
            midpoint = (min_mean + max_mean) / Decimal(2)
            candidates.append((abs(normalized_mean_log - midpoint), active_count))

    candidates.sort(key=lambda item: item[0])
    return [active_count for _, active_count in candidates]

# -----
# Decoder
# -----

def decode_unknown_mean_voltage(
    channels: List[Channel],
    measured_voltage: Decimal,
    v_offset: Decimal,
    k_value: Decimal,
    voltage_tolerance: Decimal,
    product_relative_tolerance: Decimal,
) -> Tuple[List[Channel], Decimal, Decimal, int, Dict[str, Any]]:
    """
    Decode measured mean voltage without enumerating subsets.

    Main steps:
    1. Convert voltage to mean log value.
    2. Prune impossible active counts using min/max mean-log windows.
    3. For each candidate active count, reconstruct candidate product.
    4. Use product-tree gcd extraction to recover channels.
    5. Verify expected voltage and return.
    """
    if measured_voltage == 0:
        return [], Decimal("0"), Decimal("0"), 0, {
            "candidate_counts": 0,
            "exp_attempts": 0,
            "factor_attempts": 0,
        }

    product_tree = build_product_tree(channels)
    windows = build_count_windows(channels)

    normalized_mean_log = (measured_voltage - v_offset) / k_value
    log_tolerance = abs(voltage_tolerance / k_value)
    candidate_counts = candidate_counts_for_mean(normalized_mean_log, windows,
log_tolerance)

    best_active: List[Channel] = []

```

```

best_expected = Decimal("0")
best_error: Optional[Decimal] = None
best_count = 0

exp_attempts = 0
factor_attempts = 0
gcd_extractions = 0

for active_count in candidate_counts:
    target_log_product = normalized_mean_log * Decimal(active_count)

    exp_attempts += 1
    target_product_decimal = target_log_product.exp()
    target_product_int =
int(target_product_decimal.to_integral_value(rounding=ROUND_HALF_EVEN))

    if target_product_int <= 1:
        continue

    rounded_distance = abs(Decimal(target_product_int) - target_product_decimal)
    allowed_distance = max(Decimal("1"), abs(target_product_decimal) *
product_relative_tolerance)

    if rounded_distance > allowed_distance:
        continue

    factor_attempts += 1

    if not product_is_known_subset(product_tree, target_product_int):
        continue

    gcd_extractions += 1
    decoded = extract_channels_from_product_tree(product_tree, target_product_int)

    if len(decoded) != active_count:
        continue

    # Defensive exact reconstruction check.
    reconstructed = 1
    for channel in decoded:
        reconstructed *= channel.prime

    if reconstructed != target_product_int:
        continue

    expected_voltage = mean_voltage_for_channels(decoded, v_offset, k_value)
    error = abs(expected_voltage - measured_voltage)

```

```

        if best_error is None or error < best_error:
            best_active = decoded
            best_expected = expected_voltage
            best_error = error
            best_count = active_count

        if error <= voltage_tolerance:
            return decoded, expected_voltage, error, active_count, {
                "candidate_counts": len(candidate_counts),
                "exp_attempts": exp_attempts,
                "factor_attempts": factor_attempts,
                "gcd_extractions": gcd_extractions,
            }

    if best_error is None:
        raise ValueError(
            "No valid subset recovered from measured voltage. "
            f"Candidate active counts after pruning: {candidate_counts}"
        )

    raise ValueError(
        f"Closest recovered subset had error {best_error}, exceeding tolerance "
        f"{voltage_tolerance}. "
        f"Recovered count was {best_count}."
    )

# -----
# Accuracy estimate
# -----

def required_voltage_accuracy_for_subset(
    decoded_channels: List[Channel],
    k_value: Decimal,
) -> Dict[str, Any]:
    """
    Estimate voltage accuracy required for product rounding to give the same integer
    product.

    For active count n and product P:
        product_guess = round(exp(n * (V - V_OFFSET) / K))

    To keep product_guess rounding to P:
        abs(dV) approximately < (K / n) * ln(1 + 0.5 / P)

    The returned number is a mathematical ideal bound for the integer-product decoder.
    """
    if not decoded_channels:

```

```

        return {
            "active_count": 0,
            "product_digits": 1,
            "required_abs_voltage": Decimal("0"),
            "positive_bound": Decimal("0"),
            "negative_bound": Decimal("0"),
        }

product = 1
for channel in decoded_channels:
    product *= channel.prime

product_decimal = Decimal(product)
active_count = Decimal(len(decoded_channels))
half_over_product = Decimal("0.5") / product_decimal

positive_bound = (k_value / active_count) * (Decimal(1) + half_over_product).ln()
negative_bound = (k_value / active_count) * (-(Decimal(1) -
half_over_product).ln())
required_abs_voltage = min(positive_bound, negative_bound)

return {
    "active_count": len(decoded_channels),
    "product_digits": len(str(product)),
    "product_log10": product_decimal.log10(),
    "required_abs_voltage": required_abs_voltage,
    "positive_bound": positive_bound,
    "negative_bound": negative_bound,
}

# -----
# Output
# -----

def save_report(
    path: str | Path,
    measured_voltage: Decimal,
    expected_voltage: Decimal,
    error: Decimal,
    decoded_channels: List[Channel],
    decoded_vector: List[int],
    actual_vector: Optional[List[int]],
    v_offset: Decimal,
    k_value: Decimal,
    precision: int,
    stats: Dict[str, Any],
    accuracy: Dict[str, Any],

```

```

) -> None:
    decoded_indices = [index for index, value in enumerate(decoded_vector) if value]
    actual_indices = [index for index, value in enumerate(actual_vector) if value] if
actual_vector is not None else None

    data = {
        "decoder": "unknown_mean_voltage_decoder_product_tree",
        "model": "V = V_OFFSET + K * mean(ln(prime) for active channels)",
        "precision_digits": precision,
        "v_offset": str(v_offset),
        "k": str(k_value),
        "measured_voltage": str(measured_voltage),
        "expected_voltage_from_decoded_subset": str(expected_voltage),
        "absolute_error": str(error),
        "decoded_active_count": len(decoded_channels),
        "decoded_channels": [channel.channel for channel in decoded_channels],
        "decoded_pixels": [channel.pixel for channel in decoded_channels],
        "decoded_primes": [str(channel.prime) for channel in decoded_channels],
        "decoded_128_vector": decoded_vector,
        "decoded_active_pixel_indices": decoded_indices,
        "actual_128_vector": actual_vector,
        "actual_active_pixel_indices": actual_indices,
        "matches_actual": None if actual_vector is None else decoded_vector ==
actual_vector,
        "stats": stats,
        "accuracy": {key: str(value) for key, value in accuracy.items()},
    }

    with Path(path).open("w", encoding="utf-8") as file:
        json.dump(data, file, indent=2)

# -----
# CLI
# -----

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description="Fast product-tree decoder for unknown optical NOR subset from
mean log-prime voltage."
    )

    parser.add_argument("--map", required=True, help="Channel map .cfg or .csv")
    parser.add_argument("--measured-voltage", default=None, help="Measured mean
voltage")
    parser.add_argument("--actual-nor", default=None, help="Optional actual NOR vector
to compare against")

```

```

    parser.add_argument("--simulate-from-actual", action="store_true", help="Compute
measured voltage from --actual-nor")

    parser.add_argument("--v-offset", default="1.0", help="V_OFFSET")
    parser.add_argument("--k", default="0.1", help="K multiplier")
    parser.add_argument("--voltage-tolerance", default="0.000001", help="Allowed
voltage error")
    parser.add_argument("--product-relative-tolerance", default="1e-80", help="Rounded
product relative tolerance")
    parser.add_argument("--precision", type=int, default=1200, help="Decimal precision
digits")

    parser.add_argument("--out", default="unknown_decode_report.json", help="Output
report JSON")
    parser.add_argument("--vector-out", default="decoded_unknown_vector.txt",
help="Output decoded 128-vector txt")
    parser.add_argument("--quiet", action="store_true", help="Hide decoded
channel/pixel lists")

    return parser

def main() -> None:
    args = build_parser().parse_args()
    getcontext().prec = args.precision

    total_start = time.perf_counter()

    channels = load_map(args.map)
    precompute_logs(channels)

    v_offset = Decimal(args.v_offset)
    k_value = Decimal(args.k)
    voltage_tolerance = Decimal(args.voltage_tolerance)
    product_relative_tolerance = Decimal(args.product_relative_tolerance)

    actual_vector = load_128_vector(args.actual_nor) if args.actual_nor else None

    if args.simulate_from_actual:
        if actual_vector is None:
            raise ValueError("--simulate-from-actual requires --actual-nor")

        actual_active = active_channels_from_vector(channels, actual_vector)
        measured_voltage = mean_voltage_for_channels(actual_active, v_offset, k_value)

    elif args.measured_voltage is not None:
        measured_voltage = Decimal(str(args.measured_voltage))

```

```

else:
    raise ValueError("Supply --measured-voltage or use --simulate-from-actual with
--actual-nor")

decode_start = time.perf_counter()

decoded_channels, expected_voltage, error, recovered_count, stats =
decode_unknown_mean_voltage(
    channels=channels,
    measured_voltage=measured_voltage,
    v_offset=v_offset,
    k_value=k_value,
    voltage_tolerance=voltage_tolerance,
    product_relative_tolerance=product_relative_tolerance,
)

decode_seconds = time.perf_counter() - decode_start
total_seconds = time.perf_counter() - total_start

decoded_vector = vector_from_active_channels(decoded_channels)

Path(args.vector_out).write_text(
    " ".join(str(value) for value in decoded_vector) + "\n",
    encoding="utf-8"
)

accuracy = required_voltage_accuracy_for_subset(decoded_channels, k_value)

stats["decode_seconds"] = decode_seconds
stats["total_seconds"] = total_seconds

save_report(
    path=args.out,
    measured_voltage=measured_voltage,
    expected_voltage=expected_voltage,
    error=error,
    decoded_channels=decoded_channels,
    decoded_vector=decoded_vector,
    actual_vector=actual_vector,
    v_offset=v_offset,
    k_value=k_value,
    precision=args.precision,
    stats=stats,
    accuracy=accuracy,
)

print()
print("Product-Tree Unknown Mean-Voltage Decode")

```

```

print("-----")
print(f"Mapped channels:           {len(channels)}")
print(f"Recovered active count:     {recovered_count}")
print(f"Measured voltage:           {short_decimal(measured_voltage)} V")
print(f"Expected decoded voltage:    {short_decimal(expected_voltage)} V")
print(f"Absolute error:             {short_decimal(error)} V")
print(f"Candidate counts after prune: {stats['candidate_counts']}")
print(f"Decimal exp attempts:         {stats['exp_attempts']}")
print(f"Factor attempts:              {stats['factor_attempts']}")
print(f"Product-tree extractions:      {stats['gcd_extractions']}")
print(f"Decode time:                  {decode_seconds:.6f} s")
print(f"Total time:                    {total_seconds:.6f} s")

print()
print("Minimum Voltage Accuracy Required for Exact Prime Reconstruction")
print("Diagnostic only")
print("-----")
print(f"Prime product digits:          {accuracy['product_digits']}")

if decoded_channels:
    print(f"log10(product):              {accuracy['product_log10']:.6f}")
    print(f"Minimum voltage accuracy:      +/-")
    {short_decimal(accuracy['required_abs_voltage'])} V")
    print(f"Positive-side safe error:      +{short_decimal(accuracy['positive_bound'])} V")
    print(f"Negative-side safe error:      -")
    {short_decimal(accuracy['negative_bound'])} V")
    print()
    print("Meaning:")
    print("  This is the strict mathematical tolerance needed for the decoder")
    print("  to reconstruct the exact prime product using round(exp(kL)).")
    print("  This is useful as a diagnostic/proof metric, but it is NOT the")
    print("  practical voltage target for reconstructing the 12-lane core")
output.")
else:
    print("Empty optical output: use photodiode dark-current threshold instead of")
    print("voltage decode.")

if not args.quiet:
    print()
    print(f"Decoded active channels:        {[channel.channel for channel in")
    decoded_channels]}")
    print(f"Decoded active pixels:          {[channel.pixel for channel in")
    decoded_channels]}")

if actual_vector is not None:
    print(f"Matches actual NOR vector:      {decoded_vector == actual_vector}")

```



```
print(f"Saved decoded vector:      {args.vector_out}")
print(f"Saved report:              {args.out}")

if __name__ == "__main__":
    main()
```